

Chapter	Chapter Name	Page No.
1	Introduction And Overview 1.1. Introduction 1.2. Abstract 1.3. Problem Statement 1.4. Objectives 1.5. Existing system 1.6. Proposed System 1.7. Features 1.8 Feasibility Study	1-8
2	Literature Survey 2.1. Introduction 2.2. Need for SRS 2.3. Purpose 2.4. Scope 2.5. Over all Description 2.6. General Constraints 2.7. Specific Requirement 2.8. Functional Requirement 2.9. Performance Requirement 2.10. System Attributes 2.11. Software and Hardware Tools	9-16

3	Design of the project 3.1. System Design 3.1.1. Introduction 3.1.2. Overview 3.1.3. Logical Design 3.1.4. Physical Design 3.1.5 Architecture Diagram	17-22
4	Database Design 4.1 System Overview 4.1..1. Context Flow Diagram 4.1.2. Data Flow Diagram (with levels) 4.1.3. Entity Relationship Model 4.2. Database Design (class diagrams) 4.3. UML Diagram	23-34
5	Testing 5.1. Introduction 5.2. Types of Tests 5.3. Level of Testing 5.4. Test cases	35-38
6	Results (code &screen shots)	39-91
7	Conclusion and Future Scope 7.1. Conclusion 7.2. Future Enhancement	92-93
8	Bibliography & References	94

1. INTRODUCTION AND OVERVIEW

1.1 Introduction

The Smart Grocery Management System is a full-stack web application developed to solve practical problems in household grocery tracking and stock planning. In many homes, grocery management is still handled manually through paper notes, memory, or scattered mobile reminders. This approach often leads to duplicate purchases, missing essential items, wasted products, and poor control over expiry-sensitive goods such as milk, bread, vegetables, and dairy products. The Smart Grocery Management System addresses these problems by bringing inventory tracking, shopping support, and administrative control into one integrated platform.

The application is designed as a role-based web system. A normal user can register, log in securely, add grocery items, update quantities, mark purchases, review low-stock alerts, and receive shopping suggestions. The implemented repository also includes seller-facing browser pages and super-admin style pages, in addition to the main administrative workspace for monitoring user activity, viewing inventory records, managing products, generating reports, and supervising queue-based operations. The system therefore supports both household-level usage and broader management workflows in a structured web environment.

From an academic point of view, the project is a suitable final year software engineering submission because it includes authentication, role-based access control, frontend and backend integration, database design, validation, reporting, and testing. Each part of the system reflects practical engineering decisions rather than generic textbook features.

1.2 Abstract

The Smart Grocery Management System is a web-based application created to improve grocery tracking, household inventory control, and stock planning. The main objective of the system is to help users manage grocery items in a structured and intelligent way. The project provides facilities for user registration, secure login, role-based session handling, item management, low-stock detection, expiry-aware processing, and shopping list generation. In addition, the system includes an admin dashboard for monitoring users, products, reports, and purchase flow.

The frontend of the application is implemented using React with Tailwind CSS to provide a responsive and modern interface. The backend is implemented using Spring Boot and exposes REST APIs for authentication, inventory operations, recommendation logic, and administration. MySQL is used to store user records and grocery item data in a relational format. The architecture separates presentation, business logic, and persistence concerns so that the application remains maintainable and scalable.

One of the key strengths of the project is that it moves beyond simple item storage. When users mark products as purchased, the system can compute expected expiry values based on product or category rules. It can also identify items that are low in stock, generate a shopping list based on urgent need, and present reminders for products that are close to expiry. These features show how software can provide decision support rather than only data storage.

The management modules of the project are equally important. The main admin workspace allows authorized users to view total users, inspect products, manage categories, review reports, and handle the purchase queue. In addition, the implemented web project contains seller-facing pages and super-admin pages, which show that the architecture was expanded beyond a minimal two-role structure.

Overall, the Smart Grocery Management System demonstrates a real implemented solution that combines user convenience, administrative visibility, and data-driven grocery planning in one integrated platform.

1.3 Problem Statement

Grocery management is a regular part of daily life, but it is often handled in unstructured ways. Many users depend on handwritten notes, phone reminders, or memory to remember what has been purchased, what is still needed, and what may expire soon. This causes several common problems:

- important items are forgotten until they are urgently needed
- the same item may be bought again because there is no clear inventory record
- perishable products expire before use
- users do not know which items need immediate restocking
- no summary view exists for tracking grocery patterns over time
- administrative monitoring is absent in larger shared environments

Traditional list-based approaches fail because they treat grocery management as a one-time note rather than an ongoing inventory process. In reality, groceries have quantity, category, status, and expiry behavior. A user needs to know not only what exists, but also whether the quantity is too low, whether the item is purchased or pending, and whether it must be consumed soon.

The problem becomes even more significant when multiple users or supervisory roles are involved. In such cases, there is a need for structured access control, consistent reporting, and centralized visibility. A system that only stores item names without authentication, validation, or reporting cannot support such use cases effectively.

The Smart Grocery Management System is therefore proposed as a solution to the problem of unstructured grocery planning, stock tracking, and expiry-related waste. It transforms grocery data into useful actions through an implemented full-stack application.

1.4 Objectives

The main objectives of the Smart Grocery Management System are listed below:

1. To provide secure registration and login for users and administrators.
2. To implement role-based access control for different workspaces.
3. To enable users to add, update, and delete grocery items easily.
4. To track quantity, category, purchase status, and expiry information for each grocery item.
5. To calculate expiry dates automatically where business rules are available.
6. To identify low-stock items that require restocking.
7. To generate a shopping list based on low stock and near-expiry conditions.
8. To provide dashboard summaries for operational decision-making.
9. To allow administrators to monitor users, products, categories, reports, and purchase queues.

1.5 Existing System

The existing approach to grocery management usually falls into one of the following categories:

- manual paper-based lists
- simple mobile notes
- spreadsheet tracking
- basic shopping checklist applications

- /e-commerce carts that do not maintain household stock history

These existing approaches suffer from several limitations.

First, manual lists do not provide data persistence in a structured, searchable form. Once an item is purchased, the list is usually discarded or reset, which means the user loses any understanding of stock trends and past buying patterns.

Second, simple checklist applications do not handle item lifecycle information such as purchased state, expiry date, or recommendation priority. They help with adding text, but do not convert that text into useful operational information.

Third, most available list systems do not support role-based administration or monitoring. They are intended for individual use and do not provide centralized dashboards, category-level summaries, or report generation.

Fourth, there is usually no intelligent stock logic. Existing tools rarely identify which item is running low, which purchased item is expiring soon, or which item should be prioritized in the next shopping cycle.

1.6 Proposed System

The proposed Smart Grocery Management System is a role-based web application that stores grocery data in a structured and persistent format and transforms that data into useful actions for users and administrators.

The proposed system introduces:

- secure login and registration
- separate roles for users and admins
- inventory management with add, update, and delete operations
- stock and purchase state tracking
- expiry-aware logic for purchased items
- low-stock based and expiry-based shopping list generation
- dashboard views for summaries and decisions
- admin reporting and queue management

Unlike the existing system, the proposed system treats grocery management as a continuous inventory process. It maintains records over time, computes derived states such as low stock or expiry urgency, and offers a better workflow for day-to-day grocery control.

The system is also suitable for demonstration as a complete software engineering implementation. It includes API-based communication, frontend routing, protected browser pages, backend validation, relational storage, and modular business logic.

1.7 Features of the System

The major features of the Smart Grocery Management System are as follows.

1.7.1 Authentication and Authorization

- User registration with validation
- User login with secure password handling
- Forgot-password support
- Google login support
- Token-based session management
- Role-aware access control
- Restricted access to user, seller, admin, and super-admin routes

1.7.2 Inventory Management

- Add grocery items
- Update quantity, name, category, and purchase state
- Delete items
- View items in list form
- Search items by keyword
- Filter by category
- Filter by purchased or pending state

1.7.3 Smart Expiry Logic

- Assign expiry dates when an item is purchased
- Use item-level or category-level rules when direct expiry is not given
- Highlight items that are expiring soon
- Show urgent reminders for expiry-sensitive products

1.7.4 Shopping List Generation

- Detect low-stock items automatically
- Detect near-expiry items that need replacement planning
- Generate a buy queue with priorities
- Support user action based on stock and expiry conditions

1.7.5 Dashboard Features

- Total items summary

- Purchased items summary
- Pending items summary
- Low-stock item count
- Recommendation display
- Low-stock watchlist
- Recent activity view
- Reminder-oriented decision support

1.7.6 Admin Features

- View all users
- Monitor products across the system
- Handle purchase queue
- Manage category information
- Manage reports and category summaries
- Review product and user statistics

1.7.7 Extended Web Role Features

- Seller dashboard pages
- Seller product management pages
- Seller order and analytics pages
- Super-admin dashboard pages
- Super-admin user and seller overview pages
- Super-admin analytics pages

1.7.8 Engineering Features

- REST API integration between frontend and backend
- Structured relational storage through MySQL
- Input validation
- Clean separation of responsibilities
- Testable and modular backend services

The combination of these features makes the Smart Grocery Management System a complete and realistic project suitable for final year evaluation.

2. LITERATURE SURVEY

2.1 Introduction

The purpose of this chapter is to define the software requirements of the Smart Grocery Management System in a detailed and implementation-oriented manner. Instead of describing software concepts in an abstract way, this chapter explains what the system actually needs to do, why it is needed, what constraints it works under, and how its behavior has been shaped by real project requirements.

The Smart Grocery Management System is positioned as a practical response to household inventory inefficiency. The requirement analysis for this project was driven by observed everyday problems such as low-stock confusion, unnecessary repeat purchasing, and expiry-based waste. The system requirements were therefore selected to support usability, reliability, role separation, and meaningful data processing.

2.2 Need for the System

The need for the system arises from the absence of a structured platform for managing grocery data over time. A grocery list is not enough in a real environment because users also need to know current stock level, purchase state, expiry behavior, and upcoming shopping needs.

The system is needed for the following reasons:

- to reduce manual tracking effort
- to avoid missed purchases of essential items
- to reduce grocery waste caused by late expiry awareness
- to maintain a searchable history of household stock
- to support administrative oversight for reports and queue processing
- to provide a simple digital workflow for recurring grocery planning

In short, the need is not only for storage, but for meaningful stock intelligence.

2.3 Purpose

The purpose of the Smart Grocery Management System web platform is to:

- digitize household grocery inventory operations
- provide secure and role-based access
- support efficient daily grocery decisions
- improve visibility of stock and expiry conditions

- create a central platform for user, seller, admin, and super-admin browser workflows already built in the project

From a software engineering perspective, the purpose is also to build a maintainable web system that demonstrates full-stack design, implementation, database handling, validation, and testing.

2.4 Scope

The scope of the current implemented web system includes:

- user account creation and login
- forgot-password token generation and password reset flow
- Google-based login support where configured
- token-based session access
- landing page and browser-based navigation
- user-specific inventory operations
- category-based and search-based item retrieval
- low-stock detection
- recommendation generation
- expiry-aware reminders
- smart shopping list creation
- admin workspace for users, products, reports, and queue managementsss
- seller workspace pages for products, orders, dashboard, and analytics
- super-admin style pages for users, sellers, dashboard, and analytics

The scope does not include:

- barcode scanning
- payment gateway integration
- SMS or email notifications
- offline mobile application deployment
- direct supplier integration as a core requirement

These out-of-scope items are more suitable for future enhancements and are discussed later in the report.

2.5 Overall Description

The Smart Grocery Management System follows a three-layer web application model:

1. Presentation Layer: React frontend with Tailwind CSS
2. Application Layer: Spring Boot REST API
3. Data Layer: MySQL relational database

The frontend is responsible for displaying forms, dashboards, lists, landing-page content, and protected management pages. The backend is responsible for authentication, rule execution, filtering, expiry calculation, report generation, and access enforcement. The database is responsible for persistent storage of users and grocery items.

The overall system is event-driven from the user point of view. A user action on the frontend triggers an API call, the backend validates and processes the request, and the database stores or retrieves the relevant information. The final response is rendered back in the UI.

2.6 General Constraints

The project operates under the following general constraints:

- It is a web-based system and depends on a browser and internet-style local communication.
- The backend requires Java and Spring Boot runtime support.
- The frontend requires Node.js and a local development server for execution.
- The database depends on MySQL availability and proper configuration.
- The system currently assumes valid server connectivity between frontend and backend.
- Security is based on token authentication; therefore, token handling must remain reliable.
- The shopping list and expiry recommendations are based on implemented rules, not external IoT or retail hardware devices.

These constraints are acceptable for a final year software engineering project because they keep the scope practical while still demonstrating a realistic software solution.

2.7 Functional Requirements

The functional requirements below describe the actual behavior expected from the system.

1 User Registration

- The system shall allow a new user to create an account using username, email, and password.

- The system shall validate required fields.
- The system shall reject duplicate usernames.
- The system shall securely store the password in hashed form.

2 User Login

- The system shall allow users to log in using valid credentials.
- The system shall return a token session after successful authentication.
- The system shall block access for invalid credentials.
- The system shall preserve role information for access control.

3 Role-Based Access

- The system shall distinguish between normal user and admin roles.
- The system shall restrict admin pages to authorized admin accounts only.
- The system shall restrict user pages to authenticated user accounts only.

4 Inventory Creation

- The system shall allow a logged-in user to add a grocery item.
- The item shall contain a name, category, quantity, and purchase state.
- The system shall attach the item to the logged-in user.

5 Inventory Update

- The system shall allow the user to update item properties.
- The user shall be able to change quantity and purchase state.
- The system shall recalculate expiry information when relevant.

6 Inventory Deletion

- The system shall allow a user to remove a grocery item.
- The deletion shall affect only items owned by that user.

7 Filtering and Search

- The system shall allow users to search items by name or category.
- The system shall allow filtering by purchased and pending states.
- The system shall allow category-based filtering.

8 Dashboard Summary

- The system shall show total items.
- The system shall show purchased item count.

- The system shall show pending item count.
- The system shall show low-stock item count.
- The system shall show recommendation and reminder sections.

9 Low-Stock Detection

- The system shall detect items whose quantity is at or below the defined threshold.
- Such items shall appear in low-stock outputs and shopping suggestions where applicable.

10 Expiry Calculation

- When a purchased item is added or updated, the system shall store an expiry date.
- If an expiry date is not provided, the system shall derive one from product or category rules where possible.
- The system shall identify expired, same-day, and near-expiry items.

11 Shopping List Generation

- The system shall generate shopping list items when quantity is low or expiry is near.
- The system shall assign priority labels to shopping suggestions.
- The shopping list shall be user-specific.

12 Recommendations

- The system shall generate recommendations using stock and purchase behavior.
- Recommendations shall include item name, category, reason, and priority.

13 Forgot Password Support

- The system shall allow users to request a password reset token.
- The system shall generate a time-limited token for password reset.
- The system shall allow password replacement through the reset flow.

14 Google Login Support

- The system shall support Google credential-based login where configuration is available.
- The system shall create or link user accounts through verified Google identity data.

15 Admin Dashboard

- The system shall allow admin users to view total users, products, categories, and low-stock products.

- The system shall provide a centralized management dashboard for system-level visibility.

16 Admin User Management

- The system shall allow admins to view user details.
- The system shall allow role updates within defined safety rules.
- The system shall allow controlled deletion of user accounts.

17 Admin Product Management

- The system shall allow admins to view products across the platform.
- The system shall allow edits to quantity and product properties where required.
- The system shall allow deletion of product records.

18 Admin Reports

- The system shall generate category-level reports.
- The system shall show purchased, pending, and expiring-soon product counts.
- The system shall display top categories and breakdown data.

19 Purchase Queue Handling

- The system shall list pending product purchases for admin action.
- The system shall allow purchase completion for queued items.

20 Seller and Super-Admin Web Modules

- The system shall provide seller-facing browser pages for dashboard, products, orders, and analytics.
- The system shall provide super-admin style browser pages for user, seller, dashboard, and analytics management.
- These pages shall remain route-protected based on role.

21 Error Handling

- The system shall return readable validation messages.
- The frontend shall display meaningful error messages to the user.
- Unauthorized access shall be handled safely.

2.8 Performance Requirements

The system is not designed as a high-frequency industrial platform, but it must still meet reasonable performance expectations for web usage.

Performance requirements include:

- page interactions should feel responsive during normal local use
- API calls should return data quickly for typical inventory sizes
- filtering and dashboard summaries should complete without visible delay for regular user data
- the database should support multi-record queries efficiently
- admin pages should remain usable even when the number of users and products grows moderately

Because the current system is built for project-scale and departmental evaluation, these requirements are realistic and sufficient.

2.9 System Attributes

1 Reliability

The system stores grocery data persistently in MySQL and performs validation on important operations. This reduces inconsistent state and improves reliability.

2 Usability

The frontend is divided into clear pages such as Login, Dashboard, Inventory, Shopping List, and Admin Panel. This improves learnability and reduces navigation confusion.

3 Maintainability

The project separates controllers, services, repositories, and pages. This structure improves readability and makes future updates easier.

4 Security

Security is handled through token-based authentication, access restrictions, hashed passwords, and role-based routing.

5 Scalability

The current implementation can be extended with more roles, notifications, analytics, mobile support, and additional product intelligence.

2.10 Software Tools

Category	Tool / Technology	Role in Project
Frontend	React	Builds user interface components
Frontend	Tailwind CSS	Provides utility-based styling
Frontend	Axios	Sends HTTP requests to backend APIs
Frontend	React Router	Handles page navigation and route protection
Backend	Spring Boot	Builds REST APIs and application services
Backend	Spring Security	Supports protected access and authorization
Backend	Spring Validation	Validates request payloads
Backend	JPA / Hibernate	Maps Java entities to database tables
Security	JWT	Supports token-based session logic
Database	MySQL	Stores relational application data
Tooling	Maven Wrapper	Runs backend build and test processes
Tooling	ESLint	Checks frontend code quality
Authentication	Google OAuth	Enables user login using Google credential

2.11 Hardware Tools

The project does not require specialized hardware. The following is sufficient:

- personal computer or laptop
- at least 8 GB RAM recommended for smooth development
- Java runtime support
- Node.js runtime support
- MySQL installation
- modern browser

3. DESIGN OF THE PROJECT

3.1 System Design

The Smart Grocery Management System is designed as a web-based client-server application. The main design goal is to maintain clear separation between presentation, business logic, and storage. This separation improves maintainability and makes the project easier to test and extend.

At the highest level, the system includes:

- React frontend for user interaction
- Spring Boot backend for service logic and REST APIs
- MySQL database for persistent storage

The frontend communicates with the backend through HTTP requests. The backend validates requests, checks authorization, performs processing, and stores or fetches data from the database. The response is returned in JSON format and rendered on the frontend.

This design was selected because it reflects modern enterprise web development practices and is suitable for a final year software engineering project.

3.2 Logical Design

The logical design defines how system functions are organized from the user's point of view. The system can be divided into the following logical modules:

3.2.1 Authentication and Access Module

This module handles:

- registration
- login
- forgot-password support
- Google login support
- token session creation
- role-based route protection

It ensures that only authenticated users can access their respective workspaces.

3.2.2 Inventory Management Module

This module handles:

- item addition
- item update

- item deletion
- item listing
- filtering and search

It is the central module for user interaction with grocery records.

3.2.3 Smart Logic Module

This module handles:

- expiry date derivation
- low-stock identification
- shopping list generation
- recommendation generation

It converts stored grocery data into actionable information.

3.2.4 Dashboard Module

This module handles:

- summary metrics
- recommendation display
- low-stock watchlist
- reminder-oriented outputs

It gives the user a consolidated operational view of inventory.

3.2.5 Admin Module

This module handles:

- user management
- product management
- purchase queue
- reports
- category operations

It supports system-level supervision.

3.2.6 Seller and Super-Admin Web Modules

This portion of the web project includes:

- seller dashboard pages
- seller products page
- seller orders page

- seller analytics page
- super-admin dashboard page
- super-admin user management page
- super-admin seller management page
- super-admin analytics page

These modules show that the project evolved beyond a single user-admin browser model.

3.3 Physical Design

The physical design describes how the implemented technologies are organized in the actual project structure.

Frontend Structure

The frontend contains:

- route definitions
- protected pages
- user screens
- admin screens
- API client integration
- utility helpers for expiry and session behavior

Important user-facing web pages include:

- Login / Register
- Dashboard
- Inventory
- Shopping List
- Home / Catalog

Important management web pages include:

- Admin Dashboard
- Users
- Products
- Categories
- Reports
- Purchase Queue
- Seller Dashboard

- Seller Products
- Seller Orders
- Seller Analytics
- Super-Admin Dashboard
- Super-Admin Users
- Super-Admin Sellers
- Super-Admin Analytics

Backend Structure

The backend contains:

- controllers for route handling
- services for business logic
- repositories for database access
- DTOs for request and response transfer
- security configuration for token handling

The controller layer exposes endpoints for authentication, grocery operations, and admin operations. The service layer contains the actual logic such as item updates, low-stock filtering, expiry handling, and report generation. The repository layer uses JPA to interact with MySQL.

Database Structure

The database stores:

- user information
- grocery item data

Each grocery item is linked to a specific user, which allows user-specific inventory isolation.

3.4 Architecture Explanation

The application follows a layered architecture with request-response communication.

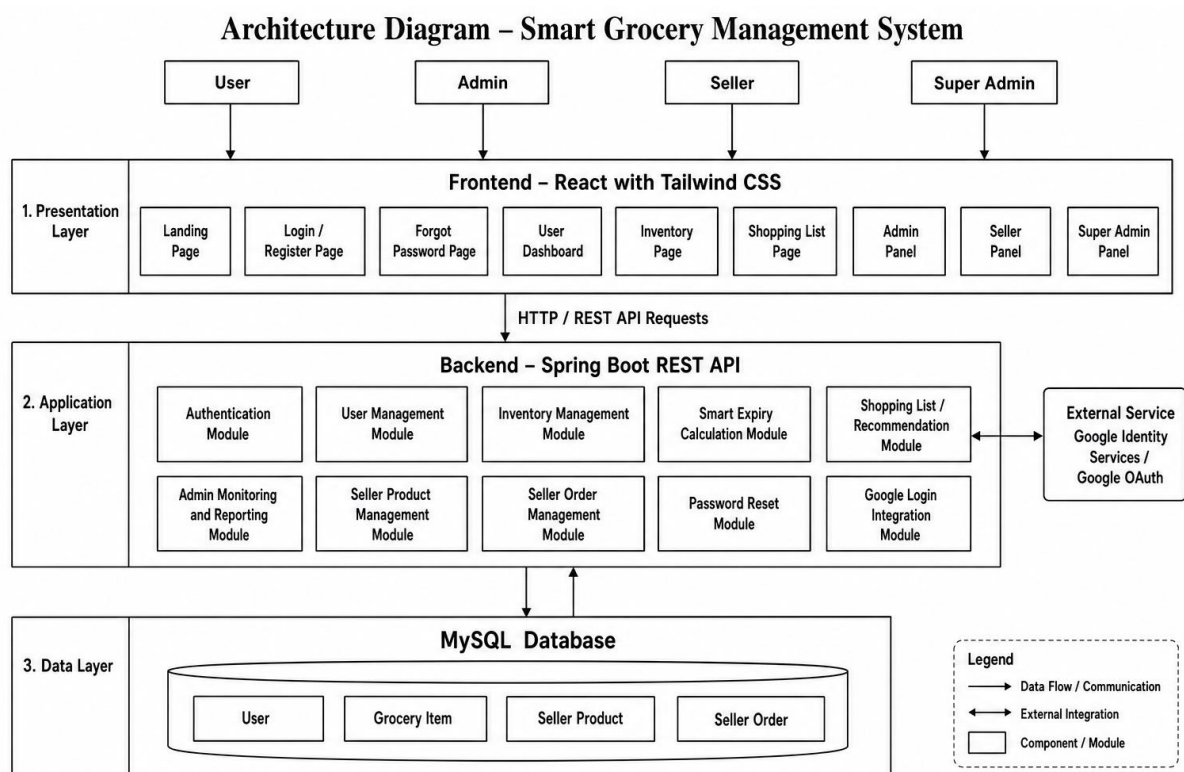
Step-by-step flow

1. The user opens a page on the React frontend.
2. The frontend sends a request to a Spring Boot REST API.
3. The backend validates the token and role if required.
4. The appropriate service method is called.

5. The service accesses the repository layer and communicates with MySQL.
6. The backend returns JSON data.
7. The frontend renders the result.

This architecture provides several advantages:

- clean separation of concerns
- easy debugging
- clear role management
- reusable API design
- maintainability for future features



3.5 Design Considerations

The following design decisions were important in the project:

- role-based routing was used to separate user and admin access
- item data was made user-scoped to prevent cross-user visibility
- service logic was used to keep controllers lightweight
- shopping and expiry logic was centralized in the backend
- report generation was placed under admin endpoints

These choices make the system more realistic than a purely page-based frontend project.

3.6 Design Strengths

The current design supports:

- modular growth
- clear workflow separation
- consistent backend behavior
- strong suitability for REST-based expansion
- manageable testing strategy

Even without complex external integrations, the design is strong enough to demonstrate real full-stack engineering decisions.

4. DATABASE DESIGN

4.1 System Overview

The Smart Grocery Management System uses MySQL as its relational database. The database is designed to support secure user accounts and user-linked grocery inventory records. The structure is intentionally focused rather than unnecessarily large, because the project prioritizes correctness, maintainability, and direct support for implemented features.

The database is responsible for:

- storing registered users
- storing grocery item records
- preserving item quantities and purchase state
- storing expiry-related fields
- supporting reporting and filtering

The system uses a one-to-many relation between users and grocery items. A single user may own many grocery items, but each grocery item belongs to one user only.

4.2 Data Entities in the Project

4.2.1 User Entity

The user entity is used for authentication and access control. It stores identity and role data.

Representative fields:

- user_id
- username
- email
- password
- role

Purpose:

- identify each account uniquely
- support login
- differentiate normal users and admin accounts

4.2.2 Grocery Item Entity

The grocery item entity stores inventory information.

Representative fields:

- item_id
- name
- category
- quantity
- purchased
- expiry_date
- last_purchased_at
- user_id

Purpose:

- maintain stock records
- support recommendation logic
- support expiry reminders
- support shopping list generation

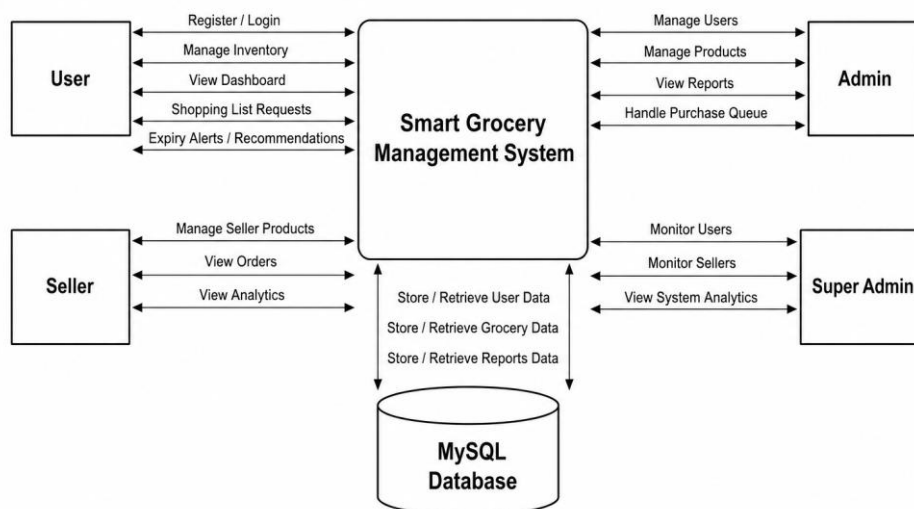
4.3 Context Flow Diagram Explanation

The context-level design treats the Smart Grocery Management System as one system boundary interacting with outside actors.

External actors include:

- user
- admin
- database

Context Diagram – Smart Grocery Management System



4.4 Data Flow Diagram - Level 0

At Level 0, the system can be viewed as a set of major processes:

1. Authentication Process
2. Inventory Management Process
3. Smart Recommendation and Shopping Process
4. Admin Monitoring and Reporting Process

Authentication Process

Input:

- registration data
- login credentials

Output:

- user account creation
- login token
- access decision

Inventory Management Process

Input:

- item add / update / delete requests

Output:

- stored item records
- updated user inventory

Smart Recommendation and Shopping Process

Input:

- item quantity
- purchase status
- expiry state

Output:

- low-stock results
- recommendations
- shopping list
- expiry reminders

Admin Monitoring and Reporting Process

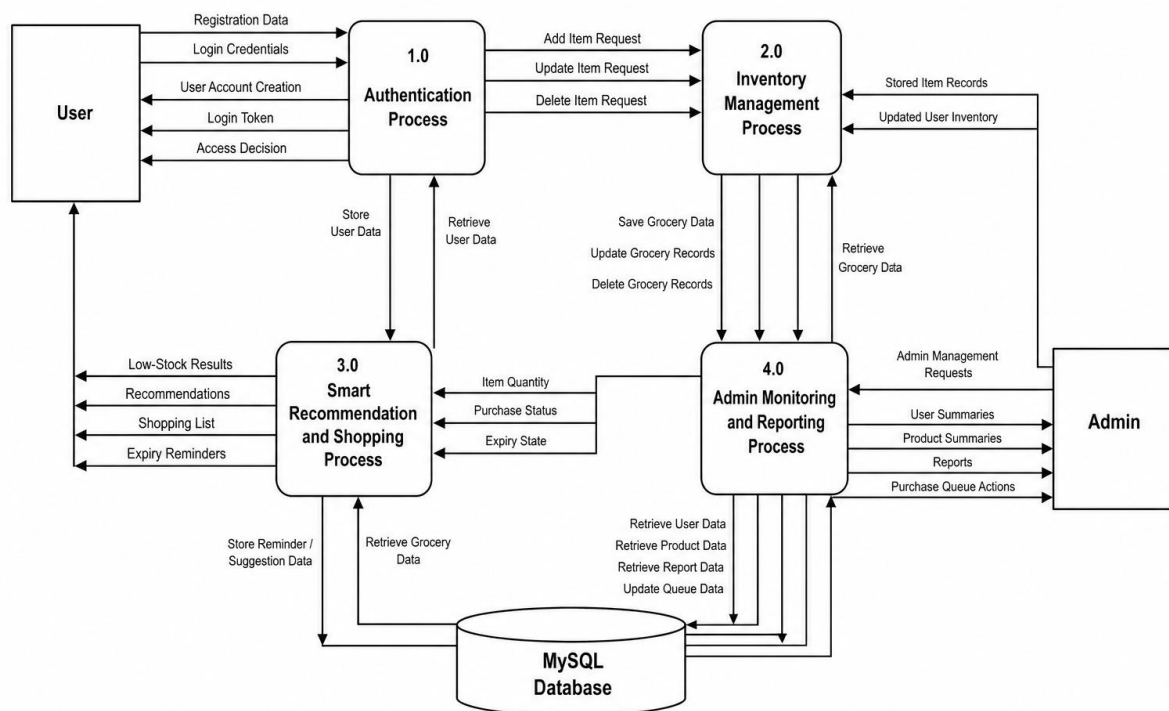
Input:

- admin management requests

Output:

- user summaries
- product summaries
- reports
- purchase queue actions

DFD Level 0 – Smart Grocery Management System



4.5 Data Flow Diagram - Level 1

Level 1 expands the major processes into more detailed operational flows.

4.5.1 Authentication Flow

- receive registration or login request
- validate input
- check existing user
- hash password when required
- generate token on successful login
- return response to frontend

4.5.2 Inventory Flow

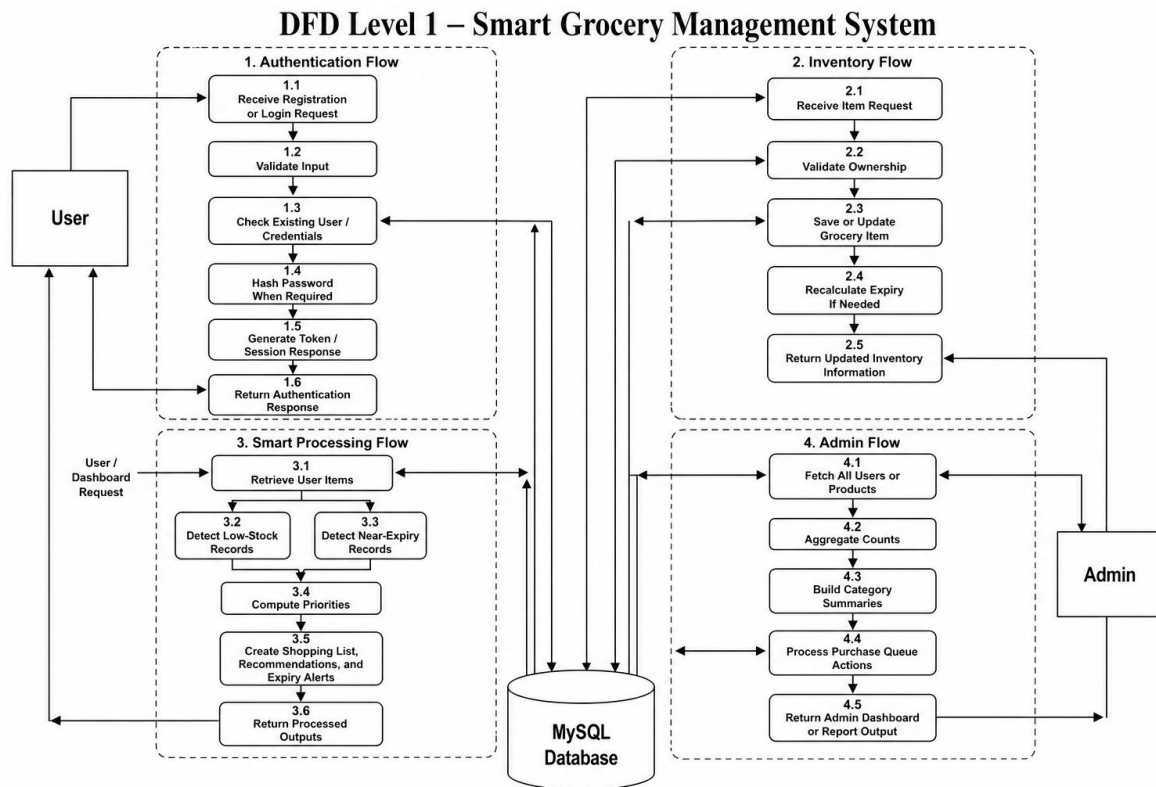
- receive item request
- validate ownership
- save or update grocery item
- recalculate expiry if needed
- return updated inventory information

4.5.3 Smart Processing Flow

- retrieve user items
- detect low-stock records
- detect near-expiry records
- compute priorities
- create shopping list and recommendations
- return processed outputs

4.5.4 Admin Flow

- fetch all users or products
- aggregate counts
- build category summaries
- process purchase queue actions
- return admin dashboard or report output



4.6 Data Flow Diagram - Level 2

Level 2 explains detailed internal decisions for selected processes.

4.6.1 Expiry Calculation Logic

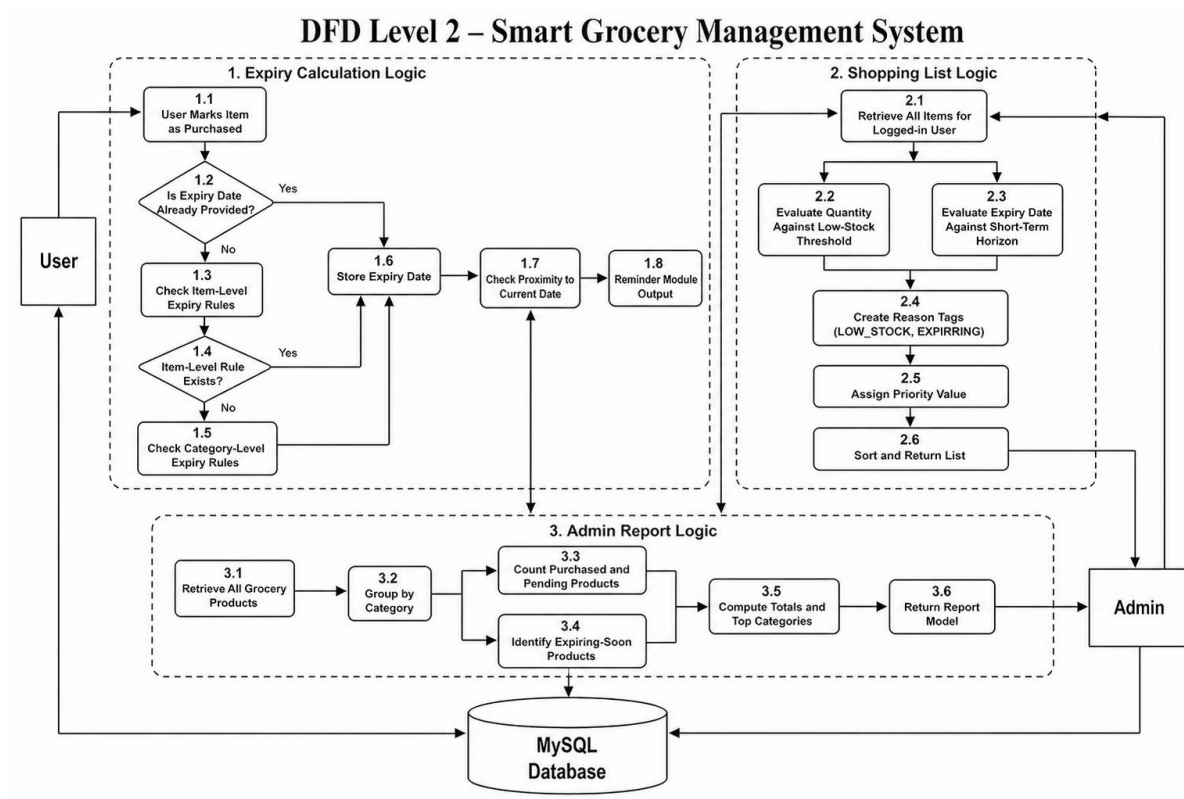
- user marks item as purchased
- system checks whether expiry date is already provided
- if not provided, system checks item-level expiry rules
- if item-level rules do not exist, category-level rules are used
- expiry date is stored
- reminder module later checks proximity to current date

4.6.2 Shopping List Logic

- retrieve all items for logged-in user
- evaluate quantity against low-stock threshold
- evaluate expiry date against short-term horizon
- create reason tags such as LOW_STOCK and EXPIRING
- assign priority value
- sort and return list

4.6.3 Admin Report Logic

- retrieve all grocery products
- group by category
- count purchased and pending products
- identify expiring-soon products
- compute totals and top categories
- return report model



4.7 Entity Relationship Model

The ER model is centered on the relationship between users and grocery items.

Main Relationship

- One user can have many grocery items.
- One grocery item belongs to one user.

This relation is sufficient for the implemented scope because item-level ownership is central to the system.

Conceptual Relationship Description

User

-> registers and logs in

-> owns grocery items

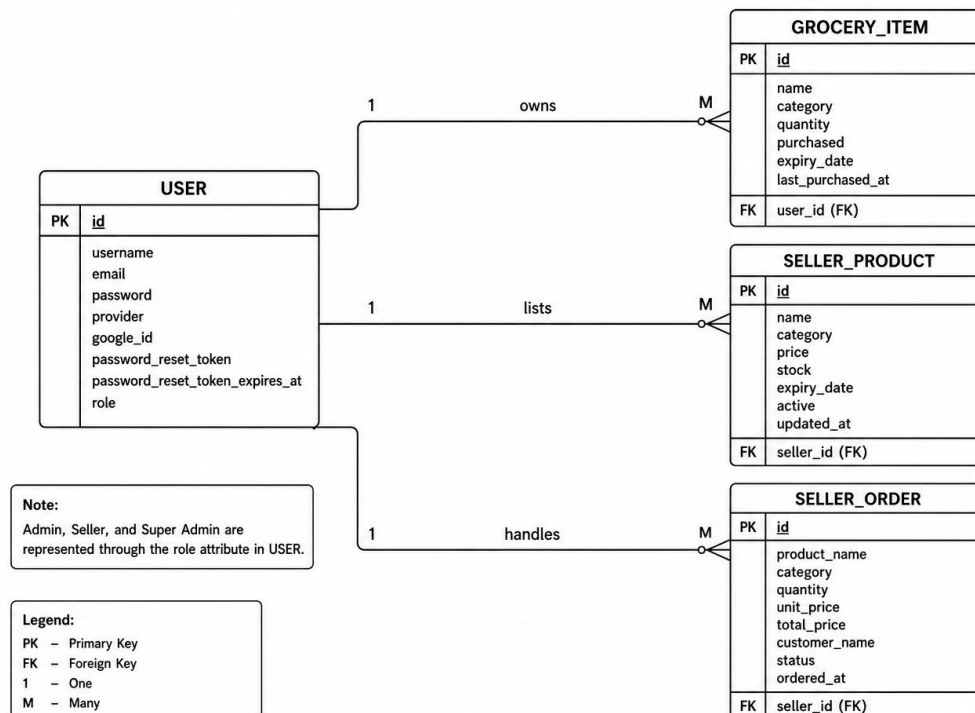
Grocery Item

-> belongs to one user

-> stores inventory state

-> drives recommendation and expiry logic

ER Diagram – Smart Grocery Management System



4.8 Class Diagram Description

The class-level design of the backend can be understood through the following major classes:

- User
- Grocery Item
- AUTH Controller
- Grocery Controller
- Admi Controller
- Auth Service
- Grocery Service
- Admin Service
- User Repository
- Grocery Repository

User Class

Stores:

- username
- email
- password
- role

Grocery Item Class

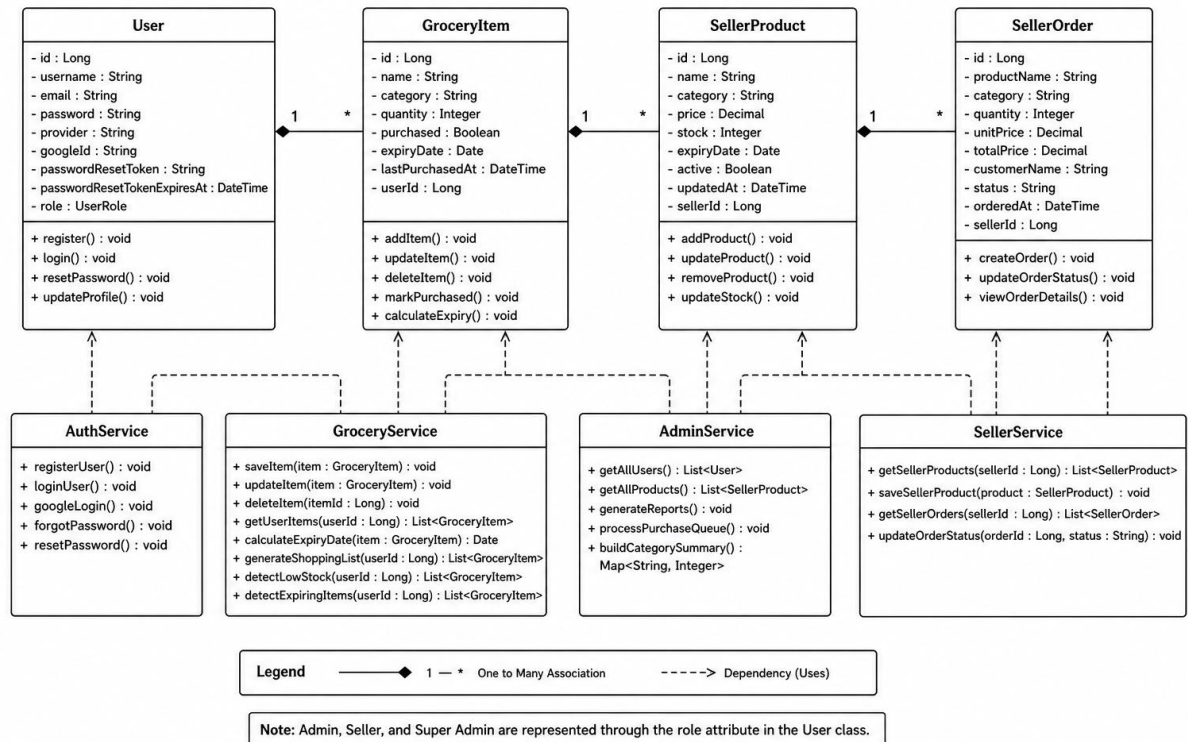
Stores:

- item details
- category
- quantity
- purchase status
- expiry information
- reference to user

Service Classes

Service classes implement business logic instead of placing it in controllers. This improves maintainability and keeps route code clean.

Class Diagram – Smart Grocery Management System



4.9 UML Diagram Description

The UML view of the system reflects the interaction between actors and modules.

User Use Cases

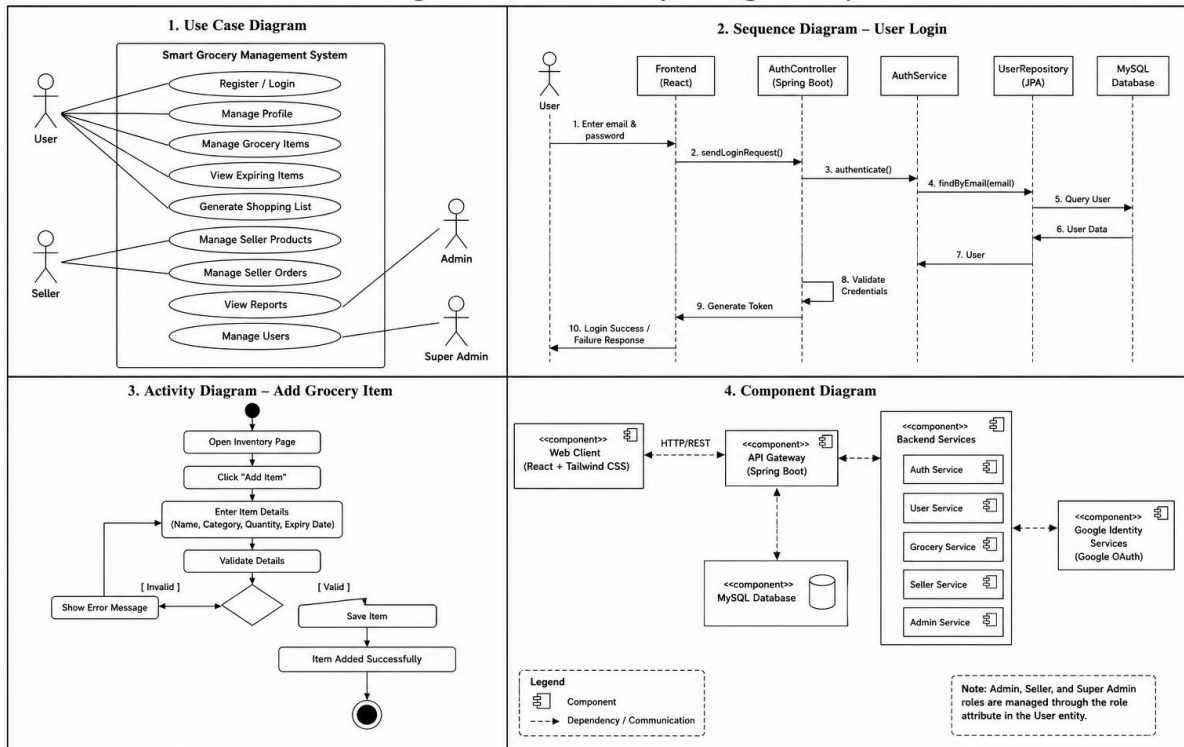
- register
- log in
- add item
- update item
- delete item
- view dashboard
- view shopping list
- acknowledge expiry reminder

Admin Use Cases

- log in
- view dashboard
- manage users

- manage products
- manage categories
- view reports
- handle purchase queue

UML Diagrams – Smart Grocery Management System



4.10 Table Design Summary

Users Table

Field	Type	Description
id	bigint	Primary key
username	varchar	Unique username
email	varchar	User email
password	varchar	Hashed password
role	varchar	USER or ADMIN

Grocery Items Table

Field	Type	Description
id	bigint	Primary key
name	varchar	Grocery item name
category	varchar	Item category
quantity	int	Number of units
purchased	boolean	Whether item is purchased
expiry_date	date	Expiry or expected expiry
last_purchased_at	datetime	Last purchase time
user_id	bigint	Foreign key to users table

4.11 Importance of the Database Design

The database design is important because every advanced feature in the system depends on stored data quality. The shopping list, low-stock detection, expiry reminders, and reports all rely on correct item records. If the database were not designed carefully, the application would reduce to a simple list tracker without reliable analytics or decision support.

For that reason, the database layer was kept simple but purposeful. Every stored field exists to support actual system behavior.

5. TESTING

5.1 Introduction

Testing is a critical part of the Smart Grocery Management System because the application includes authentication, role-based access, inventory operations, expiry logic, and report generation. These features must work correctly not only in isolation but also when integrated together.

5.2 Types of Testing

5.2.1 Unit Testing

Unit testing is used to verify individual service-level behavior. In this project, unit-oriented testing is important for:

- authentication logic
- recommendation logic
- expiry date calculation
- shopping list generation
- report aggregation

5.2.2 Integration Testing

Integration testing checks whether multiple layers work correctly together. Example areas include:

- controller to service communication
- service to repository flow
- API request and JSON response handling
- frontend-to-backend request-response expectations

5.2.3 Functional Testing

Functional testing verifies whether business requirements are met. In this project it covers:

- user registration
- login
- item creation
- item update
- item deletion

- dashboard loading
- shopping list behavior
- admin reporting

5.2.4 Validation Testing

Validation testing checks that user input rules are enforced properly. Invalid usernames, missing required fields, and unauthorized access attempts are examples of validation concerns.

5.2.5 Security Testing

Security testing ensures:

- invalid credentials are rejected
- unauthorized page access is blocked
- role-restricted routes are protected
- user data is isolated

5.3 Levels of Testing

5.3.1 Module Level

Each important module is checked independently:

- Authentication
- Inventory
- Shopping list
- Dashboard
- Admin reporting

5.3.2 API Level

Each backend endpoint is validated for:

- request correctness
- response format
- permission requirements
- error handling

5.3.3 User Interface Level

The frontend is verified for:

- route navigation

- form handling
- message display
- page-level consistency

5.3.4 System Level

The complete workflow is observed from login through inventory interaction and admin monitoring to ensure cross-module consistency.

5.4 Test Cases

Test Case ID	Module	Test Scenario	Input Condition /	Expected Result	Status
TC01	Authentication	Register new user	Valid username, email, password	User account created successfully	Pass
TC02	Authentication	Duplicate username registration	Existing username	System rejects registration	Pass
TC03	Authentication	Valid login	Correct credentials	Token returned and user redirected	Pass
TC04	Authentication	Invalid login	Wrong password	Login rejected with error	Pass
TC05	Authorization	Access user page without login	No token	Redirect or unauthorized response	Pass
TC06	Authorization	Access admin route as user	USER role token	Access denied	Pass
TC07	Inventory	Add grocery item	Valid item data	Item stored and visible in inventory	Pass
TC08	Inventory	Update grocery item	Existing item with changed quantity	Updated values saved	Pass
TC09	Inventory	Delete grocery item	Existing item id	Item removed from inventory	Pass
TC10	Inventory	Search item by name	Search key -word present	Matching items returned	Pass

Test Case ID	Module	Test Scenario	Input Condition /	Expected Result	Status
TC11	Inventory	Filter by category	Category selected	Only matching category items shown	Pass
TC12	Smart Logic	Low-stock detection	Quantity at threshold or below	Item shown in low-stock results	Pass
TC13	Smart Logic	Expiry calculation	Purchased item without explicit expiry	Expiry date generated using rules	Pass
TC14	Shopping List	Build shopping queue	Low stock or near expiry item exists	Item appears in shopping list	Pass
TC15	Dashboard	Load dashboard summary	Authenticated user	Summary cards and sections displayed	Pass
TC16	Dashboard	View expiry alerts	Purchased item near expiry	Reminder displayed	Pass
TC17	Admin	View users list	Admin login	User summary data shown	Pass
TC18	Admin	View products list	Admin login	Product records displayed	Pass
TC19	Admin	Generate reports	Admin login	Category-wise report data returned	Pass
TC20	Admin	Handle purchase queue	Pending queue item exists	Purchase completion processed	Pass

5.5 Testing Observations

The testing process shows that the project is functionally meaningful because important workflows depend on multiple layers working together. A login action touches validation, user lookup, password matching, token generation, and frontend storage. A shopping list result depends on quantity logic, expiry evaluation, and correct user scoping. An admin report depends on aggregated database data rather than isolated item retrieval.

6. RESULTS (screen shots)

6.1 Overview of Results

The Smart Grocery Management System produced a working full-stack application that successfully implemented the planned user and admin features. The final output is not a prototype or concept-only model. It includes real routes, pages, services, APIs, and database operations.

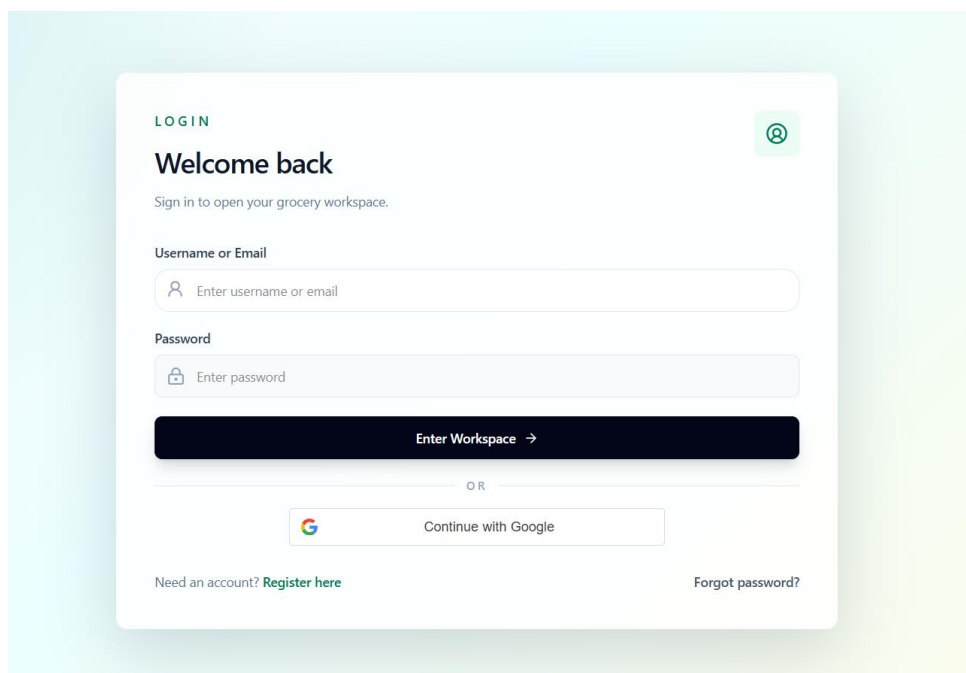
This chapter explains the major system outputs and screens. Screenshot positions are intentionally left as placeholders so they can be inserted later in the final Word version.

6.2 Login Page

The login page is the entry point for existing users and administrators. It accepts credentials and validates them through the backend authentication API. On successful login, the system issues a token session and redirects the user to the correct workspace.

Observed output:

- clean form-based input
- invalid login handling
- secure entry into the system
- role-based route access after login

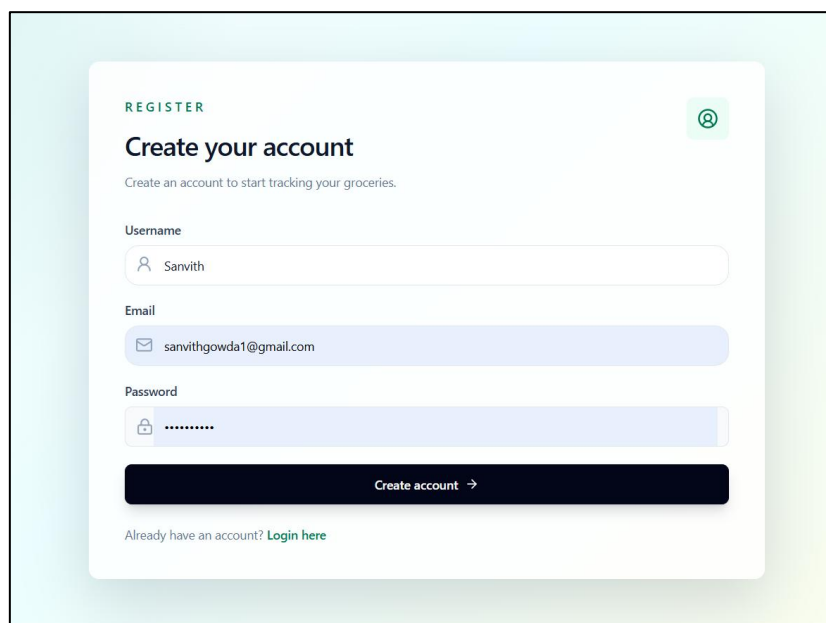


6.3 Register Page

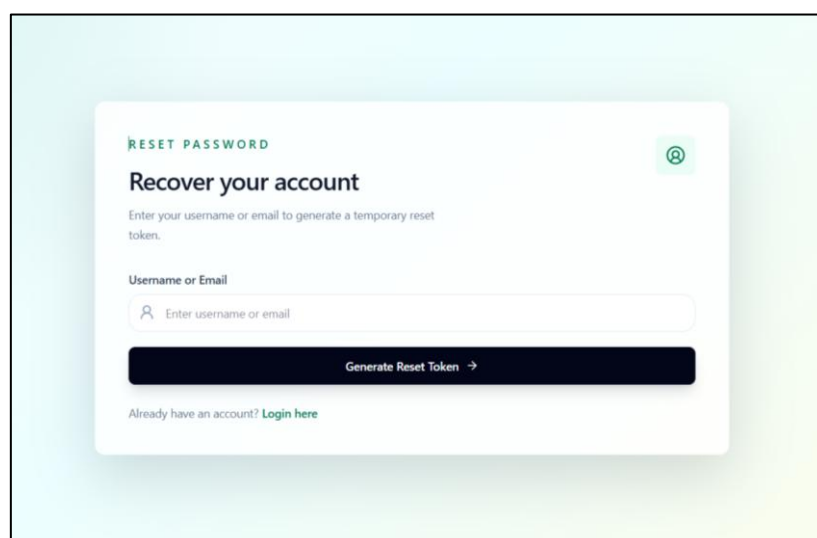
The registration page allows new users to create an account. Required data is validated before a new record is stored. Duplicate usernames are rejected to maintain account uniqueness.

Observed output:

- new account creation
- validation of required fields
- safe storage of user information



The screenshot shows a registration form titled "REGISTER" and "Create your account". The subtitle is "Create an account to start tracking your groceries." The form has three input fields: "Username" with the value "Sanvith", "Email" with the value "sanvithgowda1@gmail.com", and "Password" with masked characters. A dark blue button labeled "Create account →" is at the bottom. A link "Already have an account? Login here" is at the bottom left.



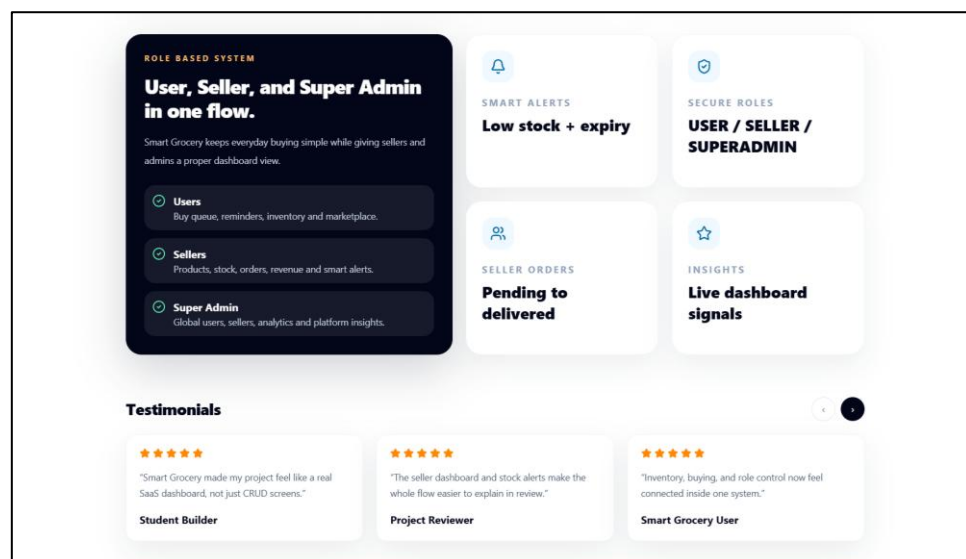
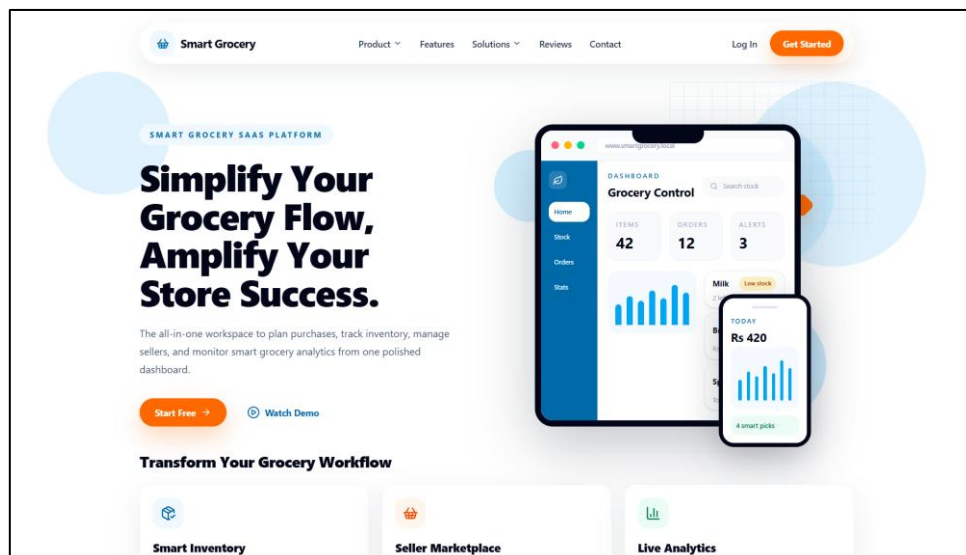
The screenshot shows a password reset form titled "RESET PASSWORD" and "Recover your account". The subtitle is "Enter your username or email to generate a temporary reset token." The form has one input field labeled "Username or Email" with the placeholder "Enter username or email". A dark blue button labeled "Generate Reset Token →" is at the bottom. A link "Already have an account? Login here" is at the bottom left.

6.4 Landing and Access Flow

The implemented web project also includes a landing page and additional access flows such as forgot-password support and Google login. These features improve the completeness of the web application and make the authentication experience more realistic than a simple single-form login process.

Observed output:

- landing page navigation
- browser-first entry into the system
- forgot-password flow
- alternate sign-in flow where configured

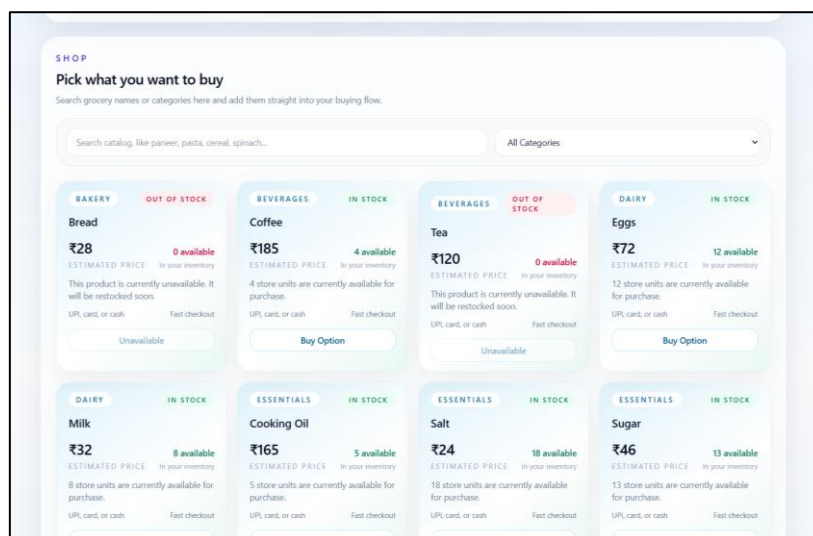
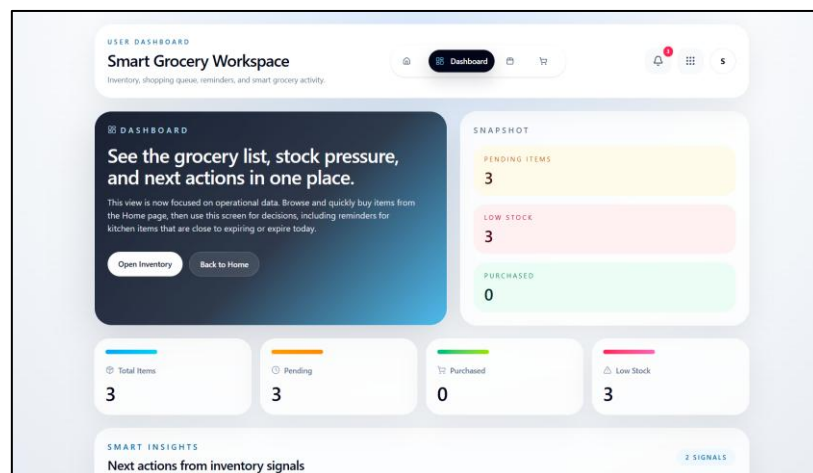


6.5 User Dashboard

The dashboard is one of the most important results of the project because it turns inventory data into a useful summary view. Instead of showing only raw item lists, the dashboard provides actionable information such as low-stock counts, recommendations, and reminders.

Observed output:

- total item count
- purchased item count
- pending item count
- low-stock item count
- smart picks
- reminder-oriented sections



6.6 Inventory Page

The inventory page allows users to create and manage their grocery records. It supports item addition, status updates, search, filtering, and deletion. This page acts as the operational center for day-to-day grocery tracking.

Observed output:

- add grocery item form
- searchable item list
- filter by category and status
- purchased / pending state update
- deletion support

The screenshot shows the 'Smart Grocery Workspace' interface. On the left, the '+ BUY ITEM' section allows users to 'Build your purchase flow' by entering an item name, selecting a category (currently '1'), and setting it as purchased. Below this is a 'DAILY PICKS' section with buttons for 'Bread BAKERY' and 'Coffee BEVERAGES'. On the right, the 'INVENTORY' section displays a list of items: 'Coffee', 'Caramel Flavored Coffee Syrup', and 'Decaffeinated Ground Coffee'. Each item shows its quantity (1), status (Pending), and expected expiry date. Action buttons for 'Mark as Bought' and 'Delete' are provided for each item.

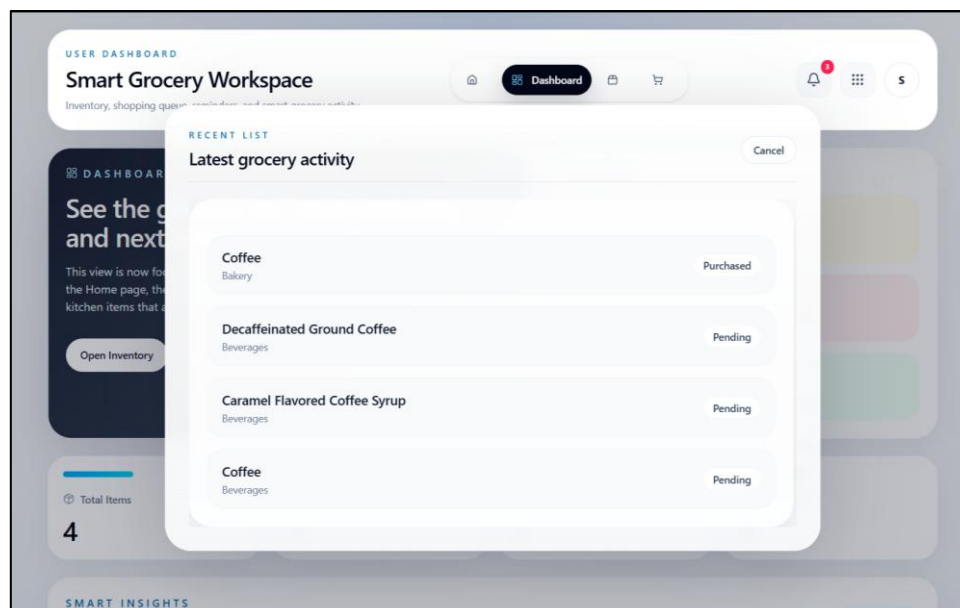
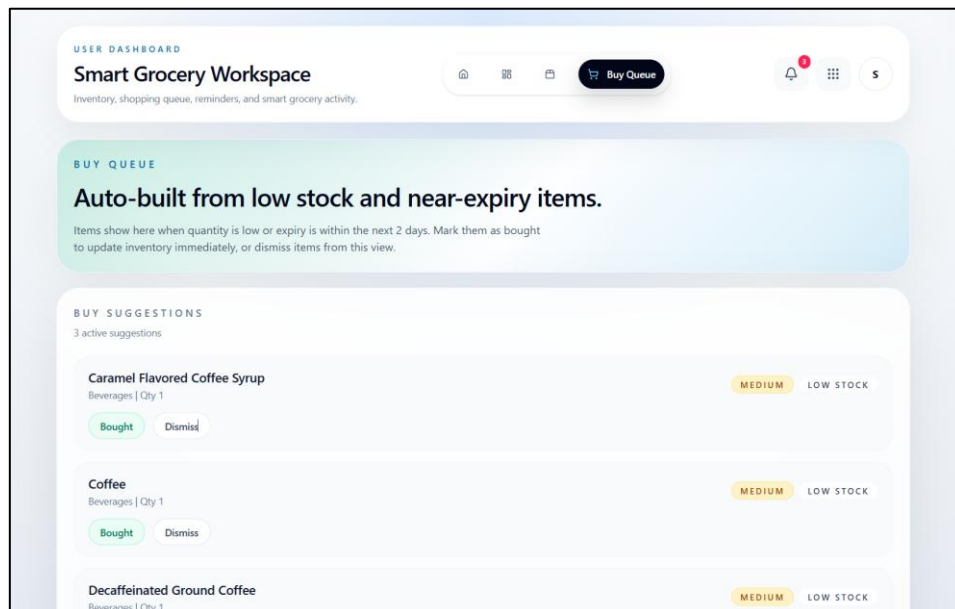
This screenshot shows the same interface as the previous one, but with the 'Select category' dropdown menu open in the '+ BUY ITEM' section. The dropdown lists categories: Bakery, Beverages, Dairy, Essentials, Fruits, Grains (highlighted), Household, Snacks, and Vegetables. The 'INVENTORY' section on the right now includes a fourth item, 'Coffee BAKERY', which has a status of 'Purchased' and a 'Move to Pending' button, along with the 'Delete' button. The other items remain the same.

6.7 Shopping List

The shopping list feature is one of the main smart outputs of the project. The list is not entered manually. Instead, it is generated using low-stock and expiry-based conditions. This gives the project a practical intelligence layer.

Observed output:

- automatically suggested shopping items
- priority labels
- explanation through reasons such as low stock or expiring soon
- better planning support for users

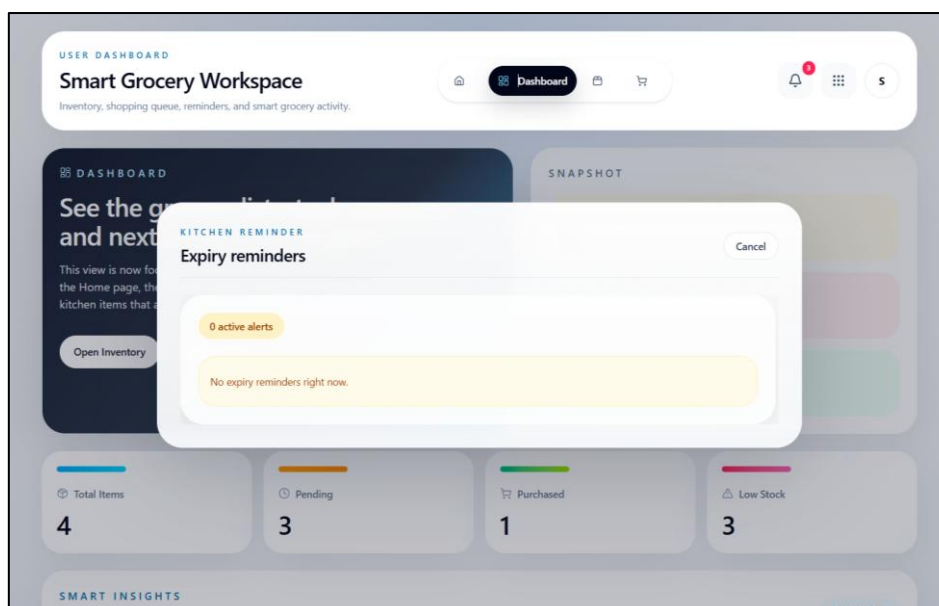
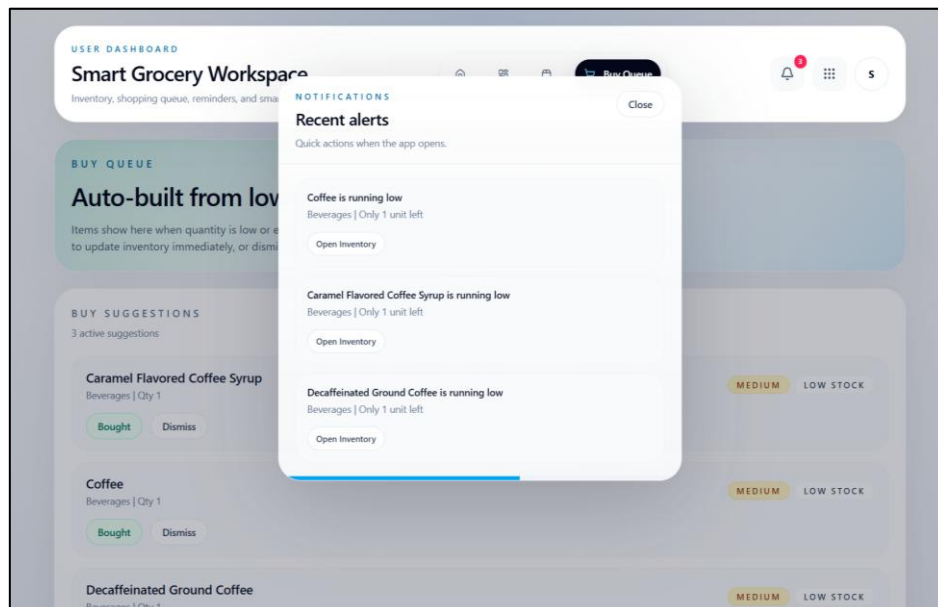


6.8 Expiry Reminder Output

The expiry reminder result is another key system output. Purchased items with relevant expiry dates are monitored by the backend logic. When an item is close to expiry, the user receives a reminder within the dashboard flow.

Observed output:

- near-expiry alerts
- same-day alerts
- attention to perishable items
- better avoidance of grocery waste

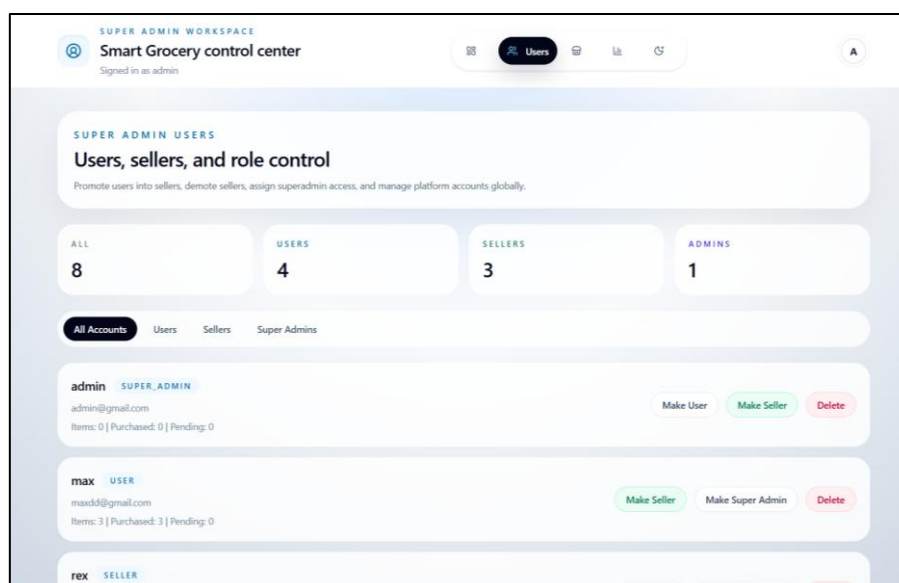
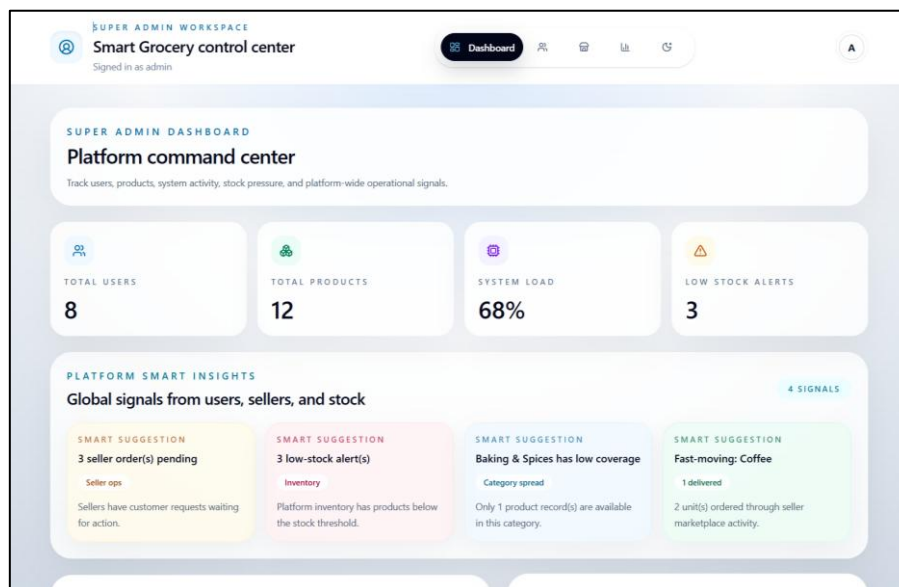


6.9 Super-Admin Web Outputs

The super-admin pages represent the highest-level management area of the project. These screens provide broader platform visibility across users, sellers, and system analytics, which makes the project more complete from an engineering perspective.

Observed output:

- super-admin dashboard summary
- super-admin user management page
- super-admin seller monitoring page
- super-admin analytics and oversight view

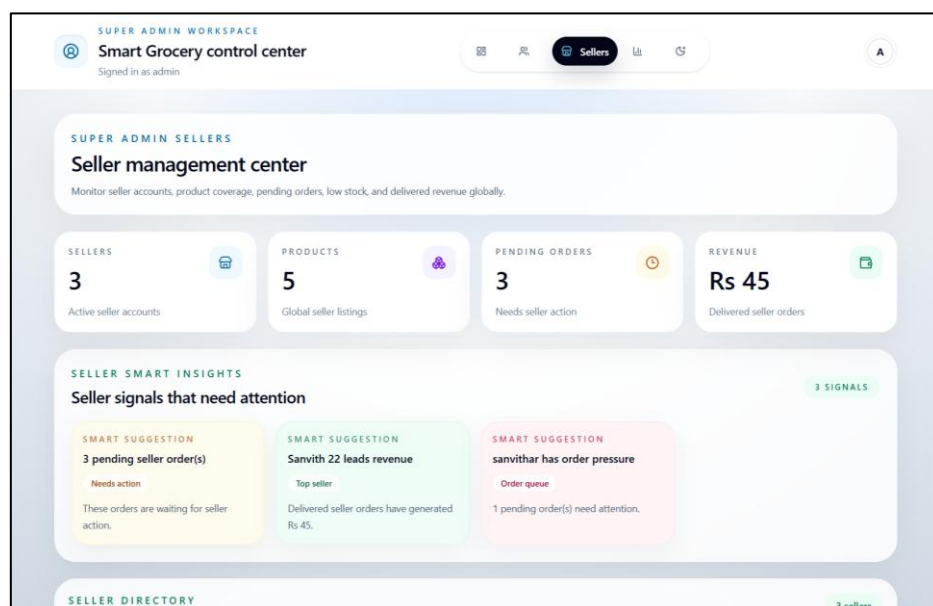
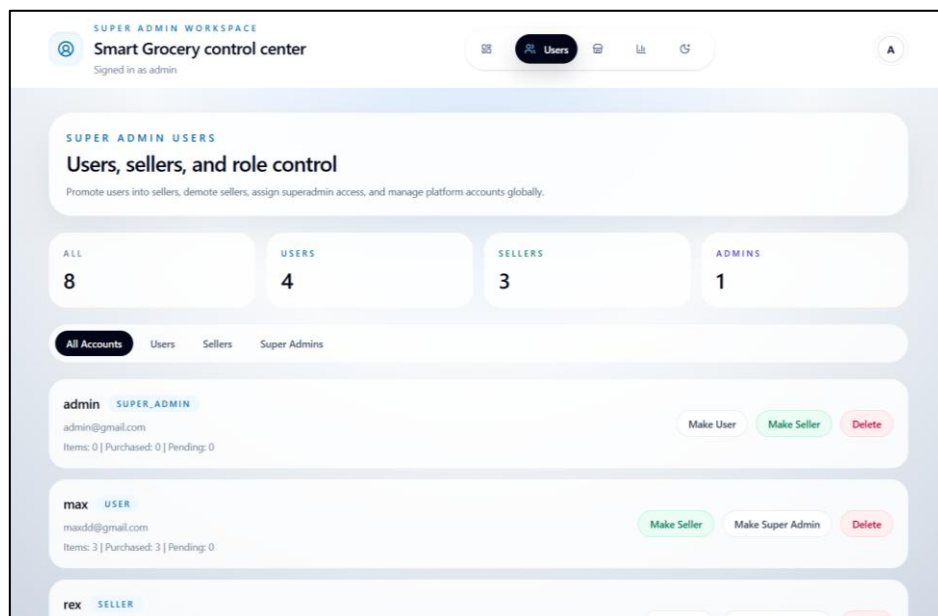


6.10 Super Admin Panel - Users and Products

The admin workspace allows deeper operational control. Admin users can monitor users, review products, and inspect category-level system state. This gives the project more depth than a simple household application.

Observed output:

- user viewing and role management
- product monitoring
- category-level tracking
- better control over system-wide data

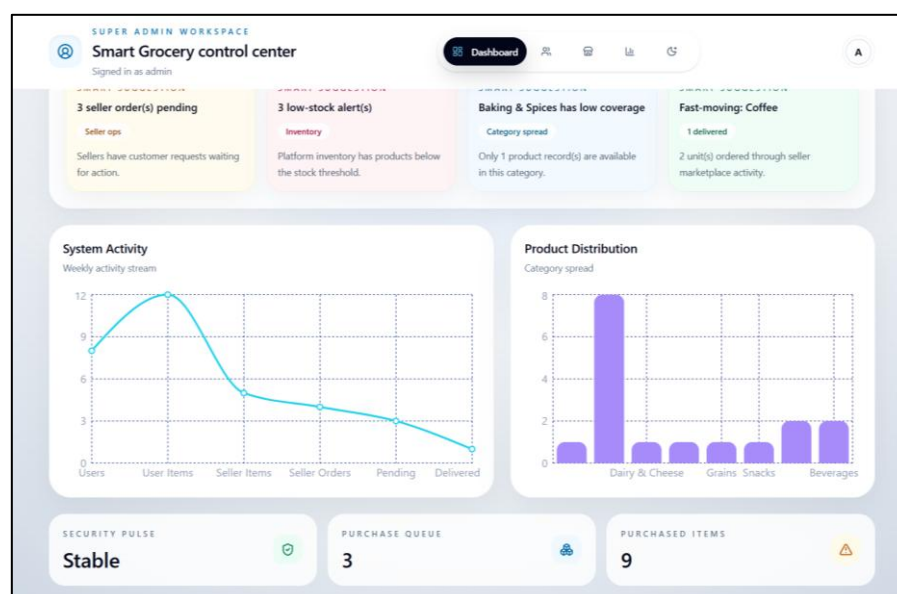
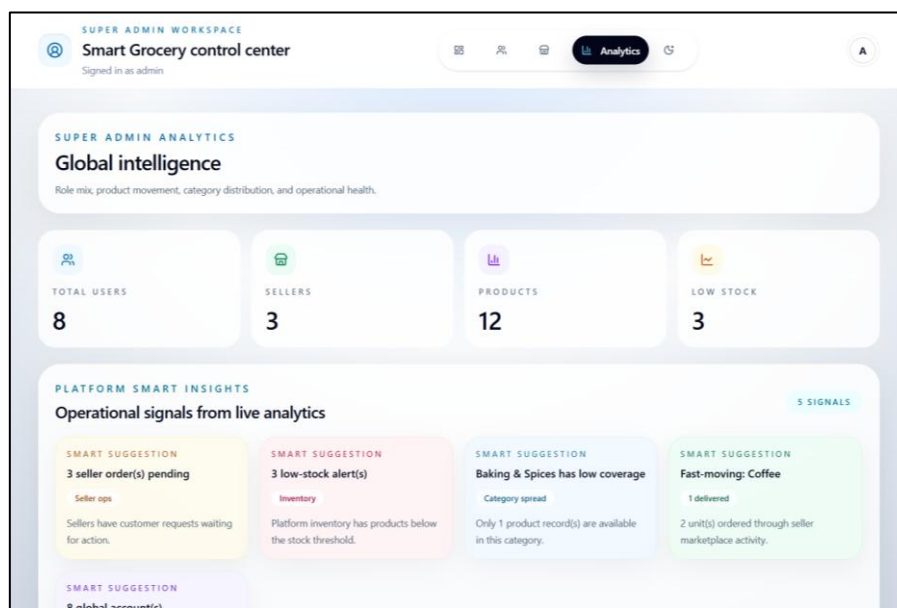


6.11 Super Admin Reports

The admin reports feature converts raw grocery data into summarized information. Reports include top categories, category-wise counts, and expiring-soon product insights.

Observed output:

- category breakdown
- top category view
- purchased and pending comparison
- reporting support for administration

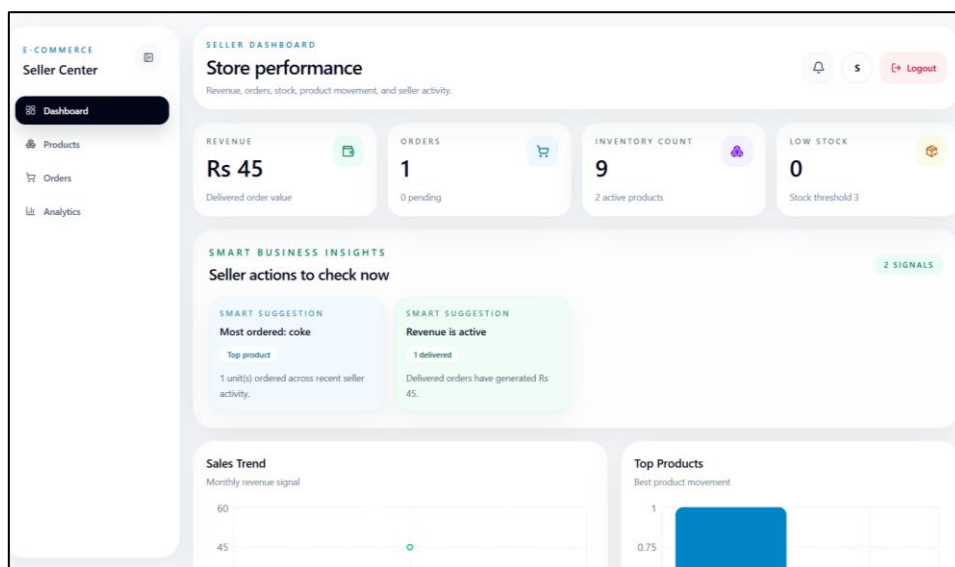


6.13 Seller Web Outputs

The implemented repository also contains seller-facing and super-admin-facing browser pages. These pages extend the project beyond a standard household-only flow and show that the role-based frontend architecture was designed to support broader management scenarios.

Observed output:

- seller dashboard and products view
- seller order tracking pages
- seller analytics page



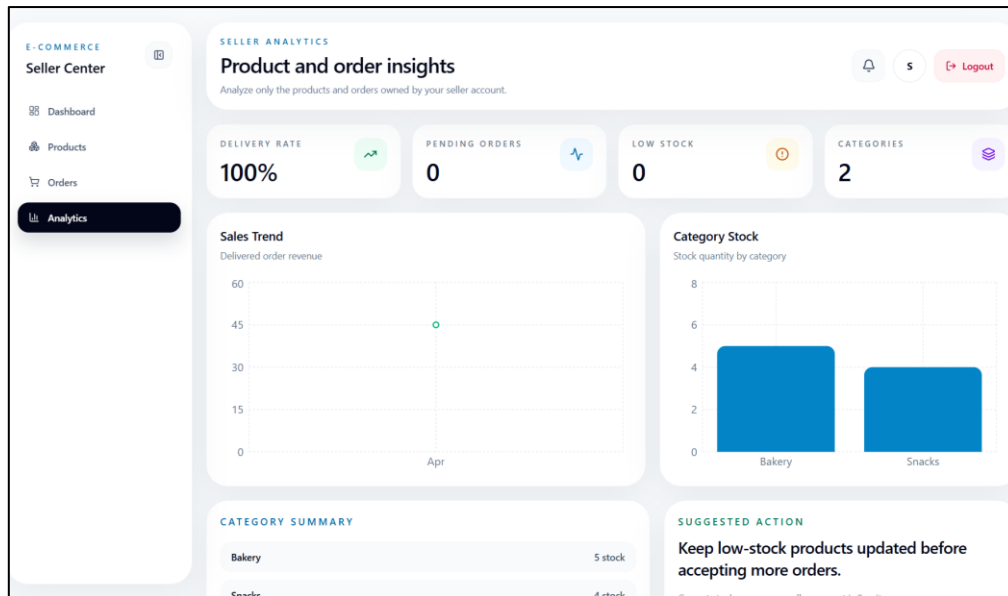
E-COMMERCE Seller Center

SELLER PRODUCTS

Manage your products
Add products, update pricing, and keep stock levels current for your seller account.

Product name Select category Price Stock dd-mm-yyyy ☒ Active

coke	SNACKS	ACTIVE	Price: Rs 45 Stock: 4 Expires: 2026-08-24	Edit Delete
Bread	BAKERY	ACTIVE	Price: Rs 28 Stock: 5 Expires: 2026-05-01	Edit Delete



E-COMMERCE Seller Center

SELLER PRODUCTS

Manage your products

Add products, update pricing, and keep stock levels current for your seller account.

Product name Select category Price Stock dd-mm-yyyy ☒ Active

coke **SNACKS** **ACTIVE**
Price: Rs 45 | Stock: 4 | Expires: 2026-08-24

Bread **BAKERY** **ACTIVE**
Price: Rs 28 | Stock: 5 | Expires: 2026-05-01

E-COMMERCE Seller Center

SELLER ORDERS

Manage order status

Move orders through pending, accepted, and delivered states.

ORDERS 1 **PENDING** 0 **ACCEPTED** 0 **DELIVERED** 1

coke **DELIVERED**
Customer: Sanwith2 | Category: Snacks | Quantity: 1
Total: ₹45

Code

```

//Config
//SecurityConfig.java

package com.example.backend.config;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.Authenticatio
nConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import jakarta.servlet.http.HttpServletResponse;
import com.example.backend.authentication.JwtFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.List;

@Configuration
public class SecurityConfig {

    @Value("${app.cors.allowed-
origins:http://localhost:3000,http://localhost:*,http://127.0.0.1:*}")
    private String allowedOrigins;

    @Autowired
    private JwtFilter jwtFilter;

    // Password Encoder
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // Authentication Manager
    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration
config) throws Exception {
        return config.getAuthenticationManager();
    }

```

```

    }

    //MAIN SECURITY CONFIG
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(csrf -> csrf.disable())
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint((request, response, authException) ->
                    response.sendError(HttpServletResponse.SC_UNAUTHORIZED,
"Unauthorized"))
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))

            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.OPTIONS, "**").permitAll()
                .requestMatchers("/auth/**").permitAll()
                .requestMatchers("/api/seller/**").hasAnyRole("SELLER",
"SUPER_ADMIN")
                .requestMatchers("/api/admin/**").hasRole("SUPER_ADMIN")
                .anyRequest().authenticated());

        http.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOriginPatterns(List.of(allowedOrigins.split(",")));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
        config.setAllowedHeaders(List.of("**"));
        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("**", config);
        return source;
    }
}

//Controller
//AuthController.java

package com.example.backend.controller;

import jakarta.validation.Valid;

```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.example.backend.dto.*;
import com.example.backend.service.AuthService;

@RestController
@RequestMapping("/auth")
public class AuthController {

    @Autowired
    private AuthService authService;

    @PostMapping("/register")
    public String register(@Valid @RequestBody RegisterRequest request) {
        return authService.register(request);
    }

    @PostMapping("/login")
    public AuthResponse login(@Valid @RequestBody AuthRequest request) {
        return authService.login(request);
    }

    @PostMapping("/forgot-password")
    public ForgotPasswordResponse forgotPassword(@Valid @RequestBody
ForgotPasswordRequest request) {
        return authService.createPasswordReset(request);
    }

    @PostMapping("/reset-password")
    public String resetPassword(@Valid @RequestBody ResetPasswordRequest request) {
        return authService.resetPassword(request);
    }

    @PostMapping("/google")
    public AuthResponse googleLogin(@Valid @RequestBody GoogleLoginRequest
request) {
        return authService.loginWithGoogle(request);
    }

    @PostMapping("/admin/login")
    public AuthResponse adminLogin(@Valid @RequestBody AuthRequest request) {
        return authService.loginAdmin(request);
    }
}

//Controller
//GroceryController.java

```

```

package com.example.backend.controller;

import com.example.backend.dto.CatalogItemResponse;
import com.example.backend.dto.ExpiryAlertResponse;
import com.example.backend.dto.GrocerySummaryResponse;
import com.example.backend.dto.RecommendationResponse;
import com.example.backend.dto.SellerOrderCreateRequest;
import com.example.backend.dto.SellerOrderResponse;
import com.example.backend.dto.SellerProductResponse;
import com.example.backend.dto.ShoppingItemDTO;
import com.example.backend.entity.GroceryItem;
import com.example.backend.service.GroceryService;
import com.example.backend.service.SellerService;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.security.Principal;
import java.util.List;

@RestController
@RequestMapping("/api/grocery")
public class GroceryController {

    @Autowired
    private GroceryService groceryService;

    @Autowired
    private SellerService sellerService;

    @GetMapping
    public List<GroceryItem> getItems(
        Principal principal,
        @RequestParam(required = false) String category,
        @RequestParam(required = false) String search,
        @RequestParam(required = false) Boolean purchased
    ) {
        return groceryService.getItemsByUsername(principal.getName(), category, search,
purchased);
    }

    @GetMapping("/categories")
    public List<String> getCategories(Principal principal) {
        return groceryService.getCategories(principal.getName());
    }

    @GetMapping("/catalog")
    public List<CatalogItemResponse> getCatalogItems(
        Principal principal,
        @RequestParam(required = false) String category,

```

```

        @RequestParam(required = false) String search
    ) {
        return groceryService.getCatalogItems(principal.getName(), category, search);
    }

    @GetMapping("/seller-products")
    public List<SellerProductResponse> getSellerProducts() {
        return sellerService.getAvailableProducts();
    }

    @PostMapping("/seller-products/{id}/order")
    public SellerOrderResponse createSellerOrder(
        Principal principal,
        @PathVariable Long id,
        @Valid @RequestBody SellerOrderCreateRequest request
    ) {
        return sellerService.createMarketplaceOrder(principal.getName(), id, request);
    }

    @GetMapping("/low-stock")
    public List<GroceryItem> getLowStockItems(Principal principal) {
        return groceryService.getLowStockItems(principal.getName());
    }

    @GetMapping("/shopping-list")
    public List<ShoppingItemDTO> getShoppingList(Principal principal) {
        return groceryService.getSmartShoppingList(principal.getName());
    }

    @GetMapping("/recommendations")
    public List<RecommendationResponse> getRecommendations(Principal principal) {
        return groceryService.getRecommendations(principal.getName());
    }

    @GetMapping("/expiry-alerts")
    public List<ExpiryAlertResponse> getExpiryAlerts(Principal principal) {
        return groceryService.getExpiryAlerts(principal.getName());
    }

    @GetMapping("/summary")
    public GrocerySummaryResponse getSummary(Principal principal) {
        return groceryService.getSummary(principal.getName());
    }

    @PostMapping
    public GroceryItem addItem(Principal principal, @Valid @RequestBody GroceryItem
item) {
        return groceryService.addItem(principal.getName(), item);
    }

```

```

    @PutMapping("/{id}")
    public GroceryItem updateItem(Principal principal, @PathVariable Long id, @Valid
    @RequestBody GroceryItem item) {
        return groceryService.updateItem(principal.getName(), id, item);
    }

    @DeleteMapping("/{id}")
    public String deleteItem(Principal principal, @PathVariable Long id) {
        groceryService.deleteItem(principal.getName(), id);
        return "Item deleted successfully";
    }

    @PostMapping("/{id}/acknowledge-expiry-alert")
    public String acknowledgeExpiryAlert(Principal principal, @PathVariable Long id) {
        groceryService.acknowledgeExpiryAlert(principal.getName(), id);
        return "Expiry alert acknowledged and item deleted";
    }
}

```

//Controller

//AdminController.java

```

package com.example.backend.controller;

import com.example.backend.dto.AdminCategoryRenameRequest;
import com.example.backend.dto.AdminCategoryResponse;
import com.example.backend.dto.AdminCatalogStockUpdateRequest;
import com.example.backend.dto.AdminDashboardResponse;
import com.example.backend.dto.AdminProductResponse;
import com.example.backend.dto.AdminReportsResponse;
import com.example.backend.dto.AdminSellerOrderResponse;
import com.example.backend.dto.AdminSellerProductResponse;
import com.example.backend.dto.AdminUserRoleUpdateRequest;
import com.example.backend.dto.AdminUserSummaryResponse;
import com.example.backend.dto.CatalogItemResponse;
import com.example.backend.entity.GroceryItem;
import com.example.backend.service.AdminService;
import jakarta.validation.Valid;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/admin")
public class AdminController {

    private final AdminService adminService;

    public AdminController(AdminService adminService) {
        this.adminService = adminService;
    }
}

```



```

    }

    @GetMapping("/dashboard")
    public AdminDashboardResponse getDashboard() {
        return adminService.getDashboard();
    }

    @GetMapping("/users")
    public List<AdminUserSummaryResponse> getUsers() {
        return adminService.getUsers();
    }

    @PutMapping("/users/{id}/role")
    public AdminUserSummaryResponse updateUserRole(
        @PathVariable Long id,
        @Valid @RequestBody AdminUserRoleUpdateRequest request
    ) {
        return adminService.updateUserRole(id, request);
    }

    @DeleteMapping("/users/{id}")
    public void deleteUser(@PathVariable Long id) {
        adminService.deleteUser(id);
    }

    @GetMapping("/products")
    public List<AdminProductResponse> getProducts() {
        return adminService.getProducts();
    }

    @PutMapping("/products/{id}")
    public AdminProductResponse updateProduct(@PathVariable Long id, @Valid
    @RequestBody GroceryItem request) {
        return adminService.updateProduct(id, request);
    }

    @DeleteMapping("/products/{id}")
    public void deleteProduct(@PathVariable Long id) {
        adminService.deleteProduct(id);
    }

    @GetMapping("/categories")
    public List<AdminCategoryResponse> getCategories() {
        return adminService.getCategories();
    }

    @PutMapping("/categories/rename")
    public List<AdminCategoryResponse> renameCategory(@Valid @RequestBody
    AdminCategoryRenameRequest request) {
        return adminService.renameCategory(request);
    }

```

```

    }

    @GetMapping("/purchase-queue")
    public List<AdminProductResponse> getPurchaseQueue() {
        return adminService.getPurchaseQueue();
    }

    @GetMapping("/seller-products")
    public List<AdminSellerProductResponse> getSellerProducts() {
        return adminService.getSellerProducts();
    }

    @GetMapping("/seller-orders")
    public List<AdminSellerOrderResponse> getSellerOrders() {
        return adminService.getSellerOrders();
    }

    @PostMapping("/purchase-queue/{id}/fulfill")
    public AdminProductResponse fulfillPurchaseQueueItem(@PathVariable Long id) {
        return adminService.fulfillPurchaseQueueItem(id);
    }

    @GetMapping("/catalog-stock")
    public List<CatalogItemResponse> getCatalogStock() {
        return adminService.getCatalogStock();
    }

    @PutMapping("/catalog-stock")
    public CatalogItemResponse updateCatalogStock(@Valid @RequestBody
AdminCatalogStockUpdateRequest request) {
        return adminService.updateCatalogStock(request);
    }

    @GetMapping("/reports")
    public AdminReportsResponse getReports() {
        return adminService.getReports();
    }
}

```

//Controller

//SellerController.java

```

package com.example.backend.controller;

import com.example.backend.dto.SellerDashboardResponse;
import com.example.backend.dto.SellerOrderResponse;
import com.example.backend.dto.SellerOrderStatusUpdateRequest;
import com.example.backend.dto.SellerProductRequest;
import com.example.backend.dto.SellerProductResponse;
import com.example.backend.service.SellerService;

```

```

import jakarta.validation.Valid;
import org.springframework.web.bind.annotation.*;

import java.security.Principal;
import java.util.List;

@RestController
@RequestMapping("/api/seller")
public class SellerController {

    private final SellerService sellerService;

    public SellerController(SellerService sellerService) {
        this.sellerService = sellerService;
    }

    @GetMapping("/dashboard")
    public SellerDashboardResponse getDashboard(Principal principal) {
        return sellerService.getDashboard(principal.getName());
    }

    @GetMapping("/products")
    public List<SellerProductResponse> getProducts(Principal principal) {
        return sellerService.getProducts(principal.getName());
    }

    @PostMapping("/products")
    public SellerProductResponse createProduct(
        Principal principal,
        @Valid @RequestBody SellerProductRequest request
    ) {
        return sellerService.createProduct(principal.getName(), request);
    }

    @PutMapping("/products/{id}")
    public SellerProductResponse updateProduct(
        Principal principal,
        @PathVariable Long id,
        @Valid @RequestBody SellerProductRequest request
    ) {
        return sellerService.updateProduct(principal.getName(), id, request);
    }

    @DeleteMapping("/products/{id}")
    public void deleteProduct(Principal principal, @PathVariable Long id) {
        sellerService.deleteProduct(principal.getName(), id);
    }

    @GetMapping("/orders")
    public List<SellerOrderResponse> getOrders(Principal principal) {

```

```

        return sellerService.getOrders(principal.getName());
    }

    @PutMapping("/orders/{id}/status")
    public SellerOrderResponse updateOrderStatus(
        Principal principal,
        @PathVariable Long id,
        @Valid @RequestBody SellerOrderStatusUpdateRequest request
    ) {
        return sellerService.updateOrderStatus(principal.getName(), id, request);
    }
}

```

//Authentication //JwtFilter.java

```

package com.example.backend.authentication;

import com.example.backend.security.JwtUtil;
import io.jsonwebtoken.JwtException;
import com.example.backend.service.CustomUserDetailsService;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class JwtFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtil jwtUtil;

    @Autowired
    private CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
response, FilterChain filterChain)
        throws ServletException, IOException {

```

```

String path = request.getServletPath();
if (path.startsWith("/auth/")) {
    filterChain.doFilter(request, response);
    return;
}

String authHeader = request.getHeader("Authorization");

if (authHeader != null && authHeader.startsWith("Bearer ")) {
    String token = authHeader.substring(7);
    try {
        String username = jwtUtil.extractUsername(token);

        if (username != null && SecurityContextHolder.getContext().getAuthentication()
== null) {
            UserDetails userDetails =
userDetailsService.loadUserByUsername(username);
            UsernamePasswordAuthenticationToken authToken =
                new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());
            authToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(authToken);
        }
    } catch (JwtException | IllegalArgumentException ex) {
        response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Invalid or
expired token");
        return;
    }
}

filterChain.doFilter(request, response);
}
}

```

//Security
//JwtUtil.java

```

package com.example.backend.security;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import jakarta.annotation.PostConstruct;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import java.nio.charset.StandardCharsets;
import java.security.Key;
import java.util.Date;

```

```

@Component
public class JwtUtil {

    @Value("${app.jwt.secret}")
    private String secret;

    private Key key;

    @PostConstruct
    public void init() {
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }

    public String generateToken(String username) {
        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60)) // 1 hour
            .signWith(key, SignatureAlgorithm.HS256)
            .compact();
    }

    public String extractUsername(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(key)
            .build()
            .parseClaimsJws(token)
            .getBody()
            .getSubject();
    }
}

```

//DTO

//AuthRequest.java

```
package com.example.backend.dto;
```

```
import jakarta.validation.constraints.NotBlank;
import lombok.Data;
```

```

@Data
public class AuthRequest {
    @NotBlank(message = "Username or email is required")
    private String username;

    @NotBlank(message = "Password is required")
    private String password;
}

```

//DTO

```
//AuthResponse.java
```

```
package com.example.backend.dto;
```

```
import lombok.AllArgsConstructor;
import lombok.Data;
```

```
@Data
@AllArgsConstructor
public class AuthResponse {
    private String token;
    private String username;
    private String role;
}
```

```
//DTO
```

```
//RegisterRequest.java
```

```
package com.example.backend.dto;
```

```
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.Data;
```

```
@Data
public class RegisterRequest {
    @NotBlank(message = "Username is required")
    @Size(min = 3, max = 40, message = "Username must be between 3 and 40 characters")
    private String username;

    @Email(message = "Email must be valid")
    @NotBlank(message = "Email is required")
    private String email;

    @NotBlank(message = "Password is required")
    @Size(min = 8, message = "Password must be at least 8 characters")
    private String password;
}
```

```
//Entity
```

```
//SellerOrder.java
```

```
package com.example.backend.entity;
```

```
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
```

```

import java.time.LocalDateTime;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "seller_orders")
public class SellerOrder {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String productName;

    private String category;

    private int quantity;

    private double unitPrice;

    private double totalPrice;

    private String customerName;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20, columnDefinition = "varchar(20)")
    private SellerOrderStatus status = SellerOrderStatus.PENDING;

    private LocalDateTime orderedAt = LocalDateTime.now();

    @JsonIgnore
    @ManyToOne
    @JoinColumn(name = "seller_id", nullable = false)
    private User seller;
}

//Repository
//UserRepository.java

package com.example.backend.repository;

import java.util.List;
import java.util.Optional;
import com.example.backend.entity.User;
import com.example.backend.entity.UserRole;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository <User,Long> {

```



```

Optional<User> findByUsername(String username);
Optional<User> findByEmail(String email);
List<User> findAllByEmailIgnoreCase(String email);
Optional<User> findByGoogleId(String googleId);
Optional<User> findByPasswordResetToken(String passwordResetToken);
long countByRole(UserRole role);
}

```

//Repository

//GroceryRepository.java

```

package com.example.backend.repository;

import com.example.backend.entity.GroceryItem;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface GroceryRepository extends JpaRepository<GroceryItem, Long> {
    List<GroceryItem> findById(Long userId);
    java.util.Optional<GroceryItem> findByIdAndUserId(Long id, Long userId);
    void deleteByUserId(Long userId);
}

```

//Service

//AuthService.java

```

package com.example.backend.service;

import java.io.IOException;
import java.security.GeneralSecurityException;
import java.time.LocalDateTime;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.UUID;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import com.google.api.client.googleapis.auth.oauth2.GoogleIdToken;
import com.google.api.client.googleapis.auth.oauth2.GoogleIdTokenVerifier;
import com.google.api.client.http.javanet.NetHttpTransport;
import com.google.api.client.json.gson.GsonFactory;

import com.example.backend.dto.*;
import com.example.backend.entity.User;

```

```

import com.example.backend.entity.UserRole;
import com.example.backend.exception.ConflictException;
import com.example.backend.exception.ResourceNotFoundException;
import com.example.backend.exception.UnauthorizedException;
import com.example.backend.repository.UserRepository;
import com.example.backend.security.JwtUtil;

@Service
public class AuthService {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @Autowired
    private JwtUtil jwtUtil;

    @Value("${app.google.client-id}")
    private String googleClientId;

    public String register(RegisterRequest request) {
        String username = request.getUsername().trim();
        String email = request.getEmail().trim().toLowerCase();

        if (userRepository.findByUsername(username).isPresent()) {
            throw new ConflictException("Username already exists");
        }
        if (!userRepository.findAllByEmailIgnoreCase(email).isEmpty()) {
            throw new ConflictException("Email already exists");
        }

        User user = new User();
        user.setUsername(username);
        user.setEmail(email);
        user.setPassword(passwordEncoder.encode(request.getPassword()));
        user.setRole(UserRole.USER);

        userRepository.save(user);

        return "User registered successfully";
    }

    public AuthResponse login(AuthRequest request) {
        User user = authenticate(request);
        return new AuthResponse(jwtUtil.generateToken(user.getUsername()),
            user.getUsername(), resolveRole(user).name());
    }

```

```

public ForgotPasswordResponse createPasswordReset(ForgotPasswordRequest request)
{
    User user = findUserByUsernameOrEmail(request.getUsernameOrEmail().trim())
        .orElseThrow(() -> new ResourceNotFoundException("Account not found"));

    String token = UUID.randomUUID().toString();
    LocalDateTime expiresAt = LocalDateTime.now().plusMinutes(15);

    user.setPasswordResetToken(token);
    user.setPasswordResetTokenExpiresAt(expiresAt);
    userRepository.save(user);

    return new ForgotPasswordResponse(
        "Password reset token generated. Use it within 15 minutes.",
        token,
        expiresAt
    );
}

public String resetPassword(ResetPasswordRequest request) {
    User user = userRepository.findByPasswordResetToken(request.getToken().trim())
        .orElseThrow(() -> new ResourceNotFoundException("Invalid reset token"));

    LocalDateTime expiresAt = user.getPasswordResetTokenExpiresAt();
    if (expiresAt == null || expiresAt.isBefore(LocalDateTime.now())) {
        user.setPasswordResetToken(null);
        user.setPasswordResetTokenExpiresAt(null);
        userRepository.save(user);
        throw new UnauthorizedException("Reset token has expired");
    }

    user.setPassword(passwordEncoder.encode(request.getNewPassword()));
    user.setProvider(user.getProvider() == null || user.getProvider().isBlank() ? "LOCAL"
: user.getProvider());
    user.setPasswordResetToken(null);
    user.setPasswordResetTokenExpiresAt(null);
    userRepository.save(user);

    return "Password reset successfully";
}

public AuthResponse loginWithGoogle(GoogleLoginRequest request) {
    GoogleIdToken.Payload payload = verifyGoogleCredential(request.getCredential());
    String googleId = payload.getSubject();
    String email = String.valueOf(payload.getEmail()).trim().toLowerCase();
    String name = String.valueOf(payload.get("name")).trim();

    User user = userRepository.findByGoogleId(googleId)
        .or(() -> findLoginUserByEmail(email))
        .orElseGet(() -> createGoogleUser(name, email, googleId));
}

```

```

        boolean changed = false;
        if (user.getGoogleId() == null || user.getGoogleId().isBlank()) {
            user.setGoogleId(googleId);
            changed = true;
        }
        if (user.getProvider() == null || user.getProvider().isBlank() ||
"LOCAL".equals(user.getProvider())) {
            user.setProvider("GOOGLE");
            changed = true;
        }
        if (changed) {
            user = userRepository.save(user);
        }

        return new AuthResponse(jwtUtil.generateToken(user.getUsername()),
user.getUsername(), resolveRole(user).name());
    }

    public AuthResponse loginAdmin(AuthRequest request) {
        User user = authenticate(request);

        if (resolveRole(user) != UserRole.SUPER_ADMIN) {
            throw new UnauthorizedException("Admin access is required");
        }

        return new AuthResponse(jwtUtil.generateToken(user.getUsername()),
user.getUsername(), resolveRole(user).name());
    }

    private GoogleIdToken.Payload verifyGoogleCredential(String credential) {
        if (googleClientId == null || googleClientId.isBlank()) {
            throw new UnauthorizedException("Google login is not configured");
        }

        try {
            GoogleIdTokenVerifier verifier = new GoogleIdTokenVerifier.Builder(
                new NetHttpTransport(),
                GsonFactory.getDefaultInstance())
                .setAudience(Collections.singletonList(googleClientId.trim()))
                .build();

            GoogleIdToken idToken = verifier.verify(credential);
            if (idToken == null) {
                throw new UnauthorizedException("Invalid Google credential");
            }

            GoogleIdToken.Payload payload = idToken.getPayload();
            if (!Boolean.TRUE.equals(payload.getEmailVerified())) {
                throw new UnauthorizedException("Google email is not verified");
            }
        }
    }

```

```

    }
    return payload;
} catch (GeneralSecurityException | IOException ex) {
    throw new UnauthorizedException("Unable to verify Google credential");
}
}

private User createGoogleUser(String name, String email, String googleId) {
    User user = new User();
    user.setUsername(buildGoogleUsername(name, email));
    user.setEmail(email);
    user.setPassword(passwordEncoder.encode("GOOGLE_LOGIN_ONLY"));
    user.setProvider("GOOGLE");
    user.setGoogleId(googleId);
    user.setRole(UserRole.USER);
    return userRepository.save(user);
}

private String buildGoogleUsername(String name, String email) {
    String base = !name.isBlank() ? name : email.substring(0, email.indexOf('@'));
    base = base.toLowerCase(Locale.ROOT).replaceAll("[^a-z0-9]+", "");
    if (base.isBlank()) {
        base = "googleuser";
    }

    String candidate = base;
    int counter = 1;
    while (userRepository.findByUsername(candidate).isPresent()) {
        candidate = base + counter;
        counter++;
    }
    return candidate;
}

private User authenticate(AuthRequest request) {
    String credential = request.getUsername().trim();

    User user = findUserByUsernameOrEmail(credential)
        .orElseThrow(() -> new ResourceNotFoundException("User not found"));

    if (!passwordEncoder.matches(request.getPassword(), user.getPassword())) {
        throw new UnauthorizedException("Invalid password");
    }

    return user;
}

private java.util.Optional<User> findUserByUsernameOrEmail(String credential) {
    String normalizedCredential = credential.toLowerCase();

```

```

        return userRepository.findByUsername(credential)
            .or(() -> findLoginUserByEmail(normalizedCredential));
    }

    private java.util.Optional<User> findLoginUserByEmail(String email) {
        List<User> matches = userRepository.findAllByEmailIgnoreCase(email);

        return matches.stream()
            .max(Comparator.comparing((User user) -> resolveRole(user) ==
                UserRole.SUPER_ADMIN)
                .thenComparing(User::getId));
    }

    private UserRole resolveRole(User user) {
        if (user.getRole() == UserRole.ADMIN) {
            return UserRole.SUPER_ADMIN;
        }

        return user.getRole() == null ? UserRole.USER : user.getRole();
    }

```

//Frontend Code

//LandingPage.jsx

```

import {
    ArrowRight,
    BarChart3,
    Bell,
    CheckCircle2,
    ChevronDown,
    Leaf,
    PackageCheck,
    PlayCircle,
    Search,
    ShieldCheck,
    ShoppingBasket,
    Star,
    Users,
} from 'lucide-react'
import { Link } from 'react-router-dom'

const featureCards = [
    {
        title: 'Smart Inventory',
        description: 'Track stock, quantity, purchase status, and expiry from one simple grocery workspace.',
        icon: PackageCheck,
        color: 'bg-sky-50 text-sky-700',
    },
    {

```

```

    title: 'Seller Marketplace',
    description: 'Order low-stock products directly from sellers and keep every request
visible.',
    icon: ShoppingBasket,
    color: 'bg-orange-50 text-orange-600',
  },
  {
    title: 'Live Analytics',
    description: 'See stock pressure, seller orders, revenue signals, and platform activity
clearly.',
    icon: BarChart3,
    color: 'bg-emerald-50 text-emerald-700',
  },
]

const dashboardRows = [
  { name: 'Milk', status: 'Low stock', tone: 'bg-amber-100 text-amber-700', value: '2 left' },
  { name: 'Bread', status: 'Seller match', tone: 'bg-sky-100 text-sky-700', value: 'Rs 28' },
  { name: 'Spinach', status: 'Expires soon', tone: 'bg-rose-100 text-rose-700', value: 'Today'
},
]

const testimonials = [
  {
    quote: 'Smart Grocery made my project feel like a real SaaS dashboard, not just CRUD
screens.',
    name: 'Student Builder',
  },
  {
    quote: 'The seller dashboard and stock alerts make the whole flow easier to explain in
review.',
    name: 'Project Reviewer',
  },
  {
    quote: 'Inventory, buying, and role control now feel connected inside one system.',
    name: 'Smart Grocery User',
  },
]

function MiniChart() {
  return (
    <div className="flex h-24 items-end gap-2 rounded-2xl bg-sky-50/70 p-3">
      {[44, 62, 52, 74, 58, 86, 70].map((height, index) => (
        <span
          key={index}
          className="flex-1 rounded-t-lg bg-sky-500 transition duration-200 hover:bg-
orange-400"
          style={{ height: `${height}%` }}
        />
      ))}
    </div>
  )
}

```

```

    </div>
  )
}

function BrowserMockup() {
  return (
    <div className="relative mx-auto max-w-xl">
      <div className="absolute -left-16 top-10 h-28 w-28 rounded-full bg-sky-200/60" />
      <div className="absolute -right-10 bottom-2 h-44 w-44 rounded-full bg-sky-500/20" />
      <div className="absolute -right-4 top-28 h-7 w-7 rotate-45 rounded-lg bg-orange-500 shadow-[0_12px_30px_rgba(249,115,22,0.35)]" />

      <div className="relative rounded-[28px] border-[10px] border-slate-950 bg-slate-950 shadow-[0_28px_90px_rgba(15,23,42,0.26)]">
        <div className="absolute left-1/2 top-0 z-10 h-5 w-28 -translate-x-1/2 rounded-b-2xl bg-slate-950" />
        <div className="overflow-hidden rounded-[18px] bg-white">
          <div className="flex items-center gap-2 border-b border-slate-100 bg-slate-50 px-4 py-3">
            <span className="h-3 w-3 rounded-full bg-rose-400" />
            <span className="h-3 w-3 rounded-full bg-amber-400" />
            <span className="h-3 w-3 rounded-full bg-emerald-400" />
            <div className="ml-3 flex-1 rounded-full bg-white px-3 py-1 text-[10px] font-semibold text-slate-400">
              www.smartgrocery.local
            </div>
          </div>

          <div className="grid min-h-[300px] grid-cols-[5.5rem_1fr] bg-white">
            <aside className="bg-sky-700 px-3 py-4 text-white">
              <div className="mb-5 flex items-center gap-2">
                <span className="flex h-7 w-7 items-center justify-center rounded-lg bg-white/15">
                  <Leaf className="h-4 w-4" />
                </span>
              </div>
              {[ 'Home', 'Stock', 'Orders', 'Stats' ].map((item, index) => (
                <div
                  key={item}
                  className={`mb-2 rounded-xl px-2 py-2 text-[10px] font-semibold ${
                    index === 0 ? 'bg-white text-sky-700' : 'text-sky-100'
                  }`}
                >
                  {item}
                </div>
              )))
            </aside>

            <section className="p-4">

```



```

    <div className="flex items-center justify-between gap-3">
      <div>
        <p className="text-[10px] font-semibold uppercase tracking-[0.24em] text-sky-700">
          Dashboard
        </p>
        <h3 className="mt-1 text-lg font-bold text-slate-950">Grocery Control</h3>
      </div>
      <div className="flex h-8 w-32 items-center gap-2 rounded-full bg-slate-50 px-3 text-[10px] text-slate-400">
        <Search className="h-3 w-3" />
        Search stock
      </div>
    </div>

    <div className="mt-4 grid grid-cols-3 gap-3">
      {[
        ['Items', '42'],
        ['Orders', '12'],
        ['Alerts', '3'],
      ].map(([label, value]) => (
        <div key={label} className="rounded-2xl border border-slate-100 bg-slate-50 p-3">
          <p className="text-[10px] uppercase tracking-[0.18em] text-slate-400">{label}</p>
          <p className="mt-1 text-2xl font-bold text-slate-950">{value}</p>
        </div>
      ))}
    </div>
  </section>
</div>
</div>
</div>
)
}

function LandingPage() {
  return (
    <main className="min-h-screen overflow-hidden bg-white text-slate-950">
      <section className="relative mx-auto min-h-screen max-w-[1180px] px-5 py-5 sm:px-8">
        <header className="relative z-10 flex items-center justify-between rounded-full bg-white/90 px-4 py-3 shadow-[0_12px_38px_rgba(15,23,42,0.08)] backdrop-blur">
          <Link to="/" className="flex items-center gap-2">
            <span className="flex h-10 w-10 items-center justify-center rounded-2xl bg-sky-50 text-sky-700">
              <ShoppingBasket className="h-5 w-5" />
            </span>
            <span className="text-lg font-bold tracking-tight">Smart Grocery</span>
          </Link>
        </header>
      </section>
    </main>
  )
}

```

```

    </Link>

    <div className="flex items-center gap-2">
      <Link to="/login" className="hidden px-3 py-2 text-sm font-semibold text-slate-
700 sm:inline-flex">
        Log In
      </Link>
      <Link
        to="/register"
        className="rounded-full bg-orange-500 px-5 py-2.5 text-sm font-bold text-
white"
      >
        Get Started
      </Link>
    </div>
  </header>

  <section className="relative z-10 grid min-h-[620px] gap-10 pt-14 lg:grid-cols-
[0.92fr_1.08fr] lg:items-center lg:pt-10">
    <div className="max-w-xl">
      <p className="mb-5 inline-flex rounded-full bg-sky-50 px-4 py-2 text-xs font-
bold uppercase tracking-[0.22em] text-sky-700">
        Smart Grocery SaaS Platform
      </p>
      <h1 className="text-4xl font-black leading-[1.05] tracking-tight text-slate-950
sm:text-5xl lg:text-[3.55rem]">
        Simplify Your Grocery Flow, Amplify Your Store Success.
      </h1>
      <p className="mt-5 max-w-lg text-base leading-7 text-slate-600">
        The all-in-one workspace to plan purchases, track inventory, manage sellers,
        and monitor smart grocery analytics from one polished dashboard.
      </p>
    </div>

    <BrowserMockup />
  </section>
</section>
</main>
)
}

```

```
export default LandingPage
```

```

//Frontend Code
//HomePage.jsx

```

```

import { useEffect, useState } from 'react'
import { Link, useNavigate } from 'react-router-dom'
import { getApiErrorMessage } from '../api/client'
import Button from '../components/ui/Button'

```

```

import Card from '../components/ui/Card'
import Input from '../components/ui/Input'
import Modal from '../components/ui/Modal'
import SmartInsightCard from '../components/ui/SmartInsightCard'
import { useToast } from '../components/ui/toast'
import {
  createGrocery,
  createSellerOrder,
  getCatalog,
  getCategories,
  getGroceries,
  getSellerProducts,
} from '../services/groceryService'
import { getNaturalExpiryDate } from '../utils/expiry'
import { buildHomeSmartBuySuggestions } from '../utils/smartSuggestions'

const AVAILABILITY_THRESHOLD = 3

const initialQuickAddDraft = {
  quantity: 1,
  purchased: true,
  expiryDate: '',
  paymentMethod: 'GPAY',
  upId: '',
  paymentVerified: false,
}

const initialSellerPaymentDraft = {
  paymentMethod: 'GPAY',
  upId: '',
  paymentVerified: false,
}

const catalogCardClasses = {
  shell: 'border-sky-100',
  chip: 'bg-white/95 text-sky-700',
  button:
    'border-sky-200 bg-white text-sky-800 hover:border-sky-300 hover:bg-sky-50',
}

const quickBuyModalStyle = {
  background: '#ffffff',
}

const homeHeroStyle = {
  background:
    'linear-gradient(135deg, rgba(219,234,254,0.92) 0%, rgba(255,255,255,0.98) 54%, rgba(240,249,255,0.92) 100%)',
}

```

```

const catalogCardStyle = {
  background:
    'linear-gradient(145deg, rgba(14,165,233,0.12), rgba(255,255,255,0.96) 58%,
    rgba(16,185,129,0.08))',
}

const stockToneByAvailability = {
  IN_STOCK: {
    label: 'In Stock',
    badge: 'bg-emerald-50 text-emerald-700',
    quantity: 'text-emerald-700',
  },
  LOW_STOCK: {
    label: 'Low Stock',
    badge: 'bg-amber-50 text-amber-700',
    quantity: 'text-amber-700',
  },
  OUT_OF_STOCK: {
    label: 'Out of Stock',
    badge: 'bg-rose-50 text-rose-700',
    quantity: 'text-rose-700',
  },
}

function HomePage() {
  const toast = useToast()
  const navigate = useNavigate()
  const [myItems, setMyItems] = useState([])
  const [catalogItems, setCatalogItems] = useState([])
  const [catalogCategories, setCatalogCategories] = useState([])
  const [sellerProducts, setSellerProducts] = useState([])
  const [sellerOrderQuantities, setSellerOrderQuantities] = useState({})
  const [sellerOrderSubmittingId, setSellerOrderSubmittingId] = useState(null)
  const [sellerOrderTarget, setSellerOrderTarget] = useState(null)
  const [sellerPaymentDraft, setSellerPaymentDraft] = useState(initialSellerPaymentDraft)
  const [catalogFilters, setCatalogFilters] = useState({ category: "", search: "" })
  const [error, setError] = useState("")
  const [quickAddTarget, setQuickAddTarget] = useState(null)
  const [quickAddDraft, setQuickAddDraft] = useState(initialQuickAddDraft)
  const [quickAddSubmitting, setQuickAddSubmitting] = useState(false)

  const loadHomeData = async () => {
    setError("")

    try {
      const [itemsData, categoriesData, catalogData, sellerProductsData] = await
Promise.all([
        getGroceries(),
        getCategories(),
        getCatalog(),

```

```

        getSellerProducts(),
    ])

    setMyItems(itemsData)
    setCatalogCategories(categoriesData)
    setCatalogItems(catalogData)
    setSellerProducts(sellerProductsData)
  } catch (requestError) {
    setError(getApiErrorMessage(requestError, 'Unable to load home data right now.'))
  }
}

useEffect(() => {
  loadHomeData()
}, [])

const loadCatalog = async (category = '', search = '') => {
  try {
    const params = {}

    if (category) {
      params.category = category
    }

    if (search) {
      params.search = search
    }

    setCatalogItems(await getCatalog(params))
  } catch (requestError) {
    setError(getApiErrorMessage(requestError, 'Unable to load catalog items right now.'))
    setCatalogItems([])
  }
}

const handleCatalogFilterChange = async (key, value) => {
  const nextFilters = { ...catalogFilters, [key]: value }
  setCatalogFilters(nextFilters)
  await loadCatalog(nextFilters.category, nextFilters.search)
}

const handleQuickAdd = async () => {
  if (!quickAddTarget) {
    return
  }

  setQuickAddSubmitting(true)
  setError('')

  try {

```

```

    await createGrocery({
      name: quickAddTarget.name,
      category: quickAddTarget.category,
      quantity: Number(quickAddDraft.quantity),
      purchased: quickAddDraft.purchased,
      expiryDate: quickAddDraft.expiryDate || null,
    })

    toast.success(`${quickAddTarget.name} added to your inventory`)
    setQuickAddTarget(null)
    setQuickAddDraft(initialQuickAddDraft)
    await loadHomeData()
  } catch (requestError) {
    setError(getApiErrorMessage(requestError, 'Could not add item right now.'))
  } finally {
    setQuickAddSubmitting(false)
  }
}

const handleSellerOrder = async (product) => {
  const quantity = Number(sellerOrderQuantities[product.id] || 1)

  if (quantity <= 0) {
    setError('Enter a valid seller order quantity.')
    return
  }

  setSellerOrderSubmittingId(product.id)
  setError("")

  try {
    await createSellerOrder(product.id, { quantity })
    toast.success(`Seller order placed for ${product.name}`)
    await loadHomeData()
  } catch (requestError) {
    setError(getApiErrorMessage(requestError, 'Could not place seller order right now.'))
  } finally {
    setSellerOrderSubmittingId(null)
  }
}

const smartSuggestions = buildHomeSmartBuySuggestions(myItems)

return (
  <div className="space-y-8">
    <section className="rounded-3xl px-6 py-8 shadow-sm" style={homeHeroStyle}>
      <p className="text-sm font-semibold uppercase tracking-[0.2em] text-sky-700">
        User Home
      </p>
      <h1 className="mt-2 text-3xl font-bold text-slate-900">

```

```

    Smart Grocery Dashboard
  </h1>
  <p className="mt-3 max-w-2xl text-sm leading-6 text-slate-600">
    Manage daily groceries, view your stock, buy missing items, and use seller listings
    from one smart home interface.
  </p>
</section>

{error && (
  <div className="rounded-2xl border border-rose-200 bg-rose-50 px-4 py-3 text-sm
text-rose-700">
    {error}
  </div>
)}

<section className="grid gap-4 md:grid-cols-3">
  <Card>
    <p className="text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">
      My Inventory
    </p>
    <h2 className="mt-2 text-3xl font-bold text-slate-900">{myItems.length}</h2>
  </Card>

  <Card>
    <p className="text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">
      Catalog Items
    </p>
    <h2 className="mt-2 text-3xl font-bold text-slate-
900">{catalogItems.length}</h2>
  </Card>

  <Card>
    <p className="text-xs font-semibold uppercase tracking-[0.18em] text-slate-500">
      Seller Products
    </p>
    <h2 className="mt-2 text-3xl font-bold text-slate-
900">{sellerProducts.length}</h2>
  </Card>
</section>

<section className="grid gap-6 lg:grid-cols-[1.1fr_0.9fr]">
  <Card>
    <div className="flex items-center justify-between">
      <div>
        <p className="text-xs font-semibold uppercase tracking-[0.18em] text-sky-700">
          Smart Buy Suggestions
        </p>
        <h3 className="mt-2 text-xl font-bold text-slate-900">
          Recommended for Purchase
        </h3>

```

```

</div>
<Link to="/shopping-list" className="text-sm font-semibold text-sky-700">
  View all
</Link>
</div>

<div className="mt-4 space-y-3">
  {smartSuggestions.map((item, index) => (
    <div
      key={` ${item.name} - ${index} `}
      className="rounded-2xl border border-slate-100 bg-white px-4 py-3"
    >
      <div className="flex items-center justify-between gap-3">
        <div>
          <p className="font-semibold text-slate-900">{item.name}</p>
          <p className="text-sm text-slate-500">{item.reason}</p>
        </div>
        <Button onClick={() => setQuickAddTarget(item)}>Buy Now</Button>
      </div>
    </div>
  ))}
</div>
</Card>

<Card>
  <p className="text-xs font-semibold uppercase tracking-[0.18em] text-sky-700">
    Filter Catalog
  </p>
  <h3 className="mt-2 text-xl font-bold text-slate-900">Browse Grocery
  Catalog</h3>

  <div className="mt-4 grid gap-3">
    <Input
      value={catalogFilters.search}
      onChange={(event) => handleCatalogFilterChange('search', event.target.value)}
      placeholder="Search grocery item"
    />

    <select
      className="rounded-2xl border border-slate-200 px-4 py-3 text-sm"
      value={catalogFilters.category}
      onChange={(event) => handleCatalogFilterChange('category', event.target.value)}
    >
      <option value="">All Categories</option>
      {catalogCategories.map((category) => (
        <option key={category} value={category}>
          {category}
        </option>
      ))}
    </select>
  </div>

```



```

    </div>
  </Card>
</section>

<section className="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
  {catalogItems.map((item) => {
    const availability = stockToneByAvailability[item.availability || 'IN_STOCK']

    return (
      <article
        key={` ${item.name}-${item.category}`}
        className={`rounded-3xl border p-5 shadow-sm ${catalogCardClasses.shell}`}
        style={catalogCardStyle}
      >
        <div className="flex items-center justify-between">
          <span className={`rounded-full px-3 py-1 text-xs font-semibold
${catalogCardClasses.chip}`}>
            {item.category}
          </span>
          <span className={`rounded-full px-3 py-1 text-xs font-semibold
${availability.badge}`}>
            {availability.label}
          </span>
        </div>

        <h3 className="mt-4 text-xl font-bold text-slate-900">{item.name}</h3>
        <p className={`mt-2 text-sm font-semibold ${availability.quantity}`}>
          Available: {item.quantity}
        </p>

        <div className="mt-4 flex gap-3">
          <Button onClick={() => setQuickAddTarget(item)}>Quick Buy</Button>
          <Button variant="secondary" onClick={() => navigate('/inventory')}>
            View Inventory
          </Button>
        </div>
      </article>
    )
  })}
</section>

<section className="space-y-4">
  <div>
    <p className="text-xs font-semibold uppercase tracking-[0.18em] text-sky-700">
      Seller Marketplace
    </p>
    <h3 className="mt-2 text-xl font-bold text-slate-900">
      Order from Sellers
    </h3>
  </div>

```

```

<div className="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
  {sellerProducts.map((product) => (
    <Card key={product.id}>
      <h4 className="text-lg font-bold text-slate-900">{product.name}</h4>
      <p className="mt-1 text-sm text-slate-500">{product.category}</p>
      <p className="mt-3 text-sm font-semibold text-slate-700">
        Stock: {product.stock}
      </p>

      <Input
        value={sellerOrderQuantities[product.id] || 1}
        onChange={(event) =>
          setSellerOrderQuantities((current) => ({
            ...current,
            [product.id]: event.target.value,
          })))
        placeholder="Quantity"
      />

      <div className="mt-4">
        <Button
          onClick={() => handleSellerOrder(product)}
          disabled={sellerOrderSubmittingId === product.id}
        >
          {sellerOrderSubmittingId === product.id ? 'Ordering...' : 'Order Product'}
        </Button>
      </div>
    </Card>
  )))
</div>
</section>

<Modal open={Boolean(quickAddTarget)} onClose={() =>
setQuickAddTarget(null)}>
  <div style={quickBuyModalStyle}>
    <h3 className="text-xl font-bold text-slate-900">
      Quick Buy {quickAddTarget?.name}
    </h3>

    <div className="mt-4 grid gap-3">
      <Input
        value={quickAddDraft.quantity}
        onChange={(event) =>
          setQuickAddDraft((current) => ({
            ...current,
            quantity: event.target.value,
          })))
      >
    </div>
  </div>

```

```

        placeholder="Quantity"
      />

      <Input
        value={quickAddDraft.expiryDate}
        onChange={(event) =>
          setQuickAddDraft((current) => ({
            ...current,
            expiryDate: event.target.value,
          })))
      />
      placeholder="Expiry Date"
    />
  </div>

  <div className="mt-5 flex gap-3">
    <Button onClick={handleQuickAdd} disabled={quickAddSubmitting}>
      {quickAddSubmitting ? 'Saving...' : 'Add to Inventory'}
    </Button>
    <Button variant="secondary" onClick={() => setQuickAddTarget(null)}>
      Cancel
    </Button>
  </div>
</div>
</Modal>
</div>
)
}

export default HomePage

//Frontend Code
//DashboardPage.jsx

import { useEffect, useState } from 'react'
import { Link, useLocation, useNavigate } from 'react-router-dom'
import { LayoutDashboard, ShoppingCart, AlertTriangle, Package, Lightbulb, Clock }
from 'lucide-react'
import { getApiErrorMessage } from '../api/client'
import Button from '../components/ui/Button'
import { SkeletonCard } from '../components/ui/Skeleton'
import SmartInsightCard from '../components/ui/SmartInsightCard'
import { useToast } from '../components/ui/toast'
import {
  acknowledgeExpiryAlert,
  getExpiryAlerts,
  getGroceries,
  getLowStock,
  getRecommendations,
  getSellerProducts,

```

```

    getSummary,
  } from '../services/groceryService'
import { buildDashboardSmartInsights, findSellerMatchesForItem } from
'../utils/smartSuggestions'

const summaryCards = [
  { key: 'totalItems', label: 'Total Items', accent: 'from-sky-500 to-cyan-400', icon: Package
  },
  { key: 'pendingItems', label: 'Pending', accent: 'from-amber-500 to-orange-400', icon:
  Clock },
  { key: 'purchasedItems', label: 'Purchased', accent: 'from-emerald-500 to-lime-400', icon:
  ShoppingCart },
  { key: 'lowStockItems', label: 'Low Stock', accent: 'from-rose-500 to-pink-400', icon:
  AlertTriangle },
]

function DashboardPage() {
  const toast = useToast()
  const location = useLocation()
  const navigate = useNavigate()
  const [summary, setSummary] = useState(null)
  const [recommendations, setRecommendations] = useState([])
  const [lowStockItems, setLowStockItems] = useState([])
  const [sellerProducts, setSellerProducts] = useState([])
  const [expiryAlerts, setExpiryAlerts] = useState([])
  const [myItems, setMyItems] = useState([])
  const [error, setError] = useState("")
  const [loading, setLoading] = useState(true)
  const [activeSection, setActiveSection] = useState("")
  const [acknowledgingItemId, setAcknowledgingItemId] = useState("")

  const dashboardSections = [
    { key: 'recommendations', label: 'Smart Picks', sublabel: 'Recommendation queue', icon:
    Lightbulb },
    { key: 'low-stock-watchlist', label: 'Action Board', sublabel: 'Low-stock watchlist', icon:
    AlertTriangle },
    { key: 'kitchen-reminders', label: 'Kitchen Reminder', sublabel: 'Expiry reminders', icon:
    Clock },
    { key: 'pending-focus', label: 'Pending Focus', sublabel: 'Items that still need action',
    icon: Package },
    { key: 'recent-list', label: 'Recent List', sublabel: 'Latest grocery activity', icon:
    ShoppingCart },
  ]

  const formatExpiryDate = (value) => {
    if (!value) {
      return ""
    }

    return new Date(`${value}T00:00:00`).toLocaleDateString()
  }

```

```

    }

    const findSellerMatches = (item) => {
      const itemName = (item.name || "").trim().toLowerCase()
      const itemCategory = (item.category || "").trim().toLowerCase()

      return findSellerMatchesForItem({ ...item, name: itemName, category: itemCategory },
sellerProducts).slice(0, 3)
    }

    const fetchDashboardData = async () => {
      const results = await Promise.allSettled([
        getSummary(),
        getRecommendations(),
        getLowStock(),
        getExpiryAlerts(),
        getGroceries(),
        getSellerProducts(),
      ])

      const [
        summaryResult,
        recommendationsResult,
        lowStockResult,
        expiryAlertsResult,
        myItemsResult,
        sellerProductsResult,
      ] = results

      const failures = results.filter((result) => result.status === 'rejected')

      return {
        failures,
        summary: summaryResult.status === 'fulfilled' ? summaryResult.value : undefined,
        recommendations: recommendationsResult.status === 'fulfilled' ?
recommendationsResult.value : undefined,
        lowStockItems: lowStockResult.status === 'fulfilled' ? lowStockResult.value :
undefined,
        expiryAlerts: expiryAlertsResult.status === 'fulfilled' ? expiryAlertsResult.value :
undefined,
        myItems: myItemsResult.status === 'fulfilled' ? myItemsResult.value : undefined,
        sellerProducts: sellerProductsResult.status === 'fulfilled' ? sellerProductsResult.value :
undefined,
      }
    }

    const applyDashboardData = (data) => {
      if (data.summary !== undefined) {
        setSummary(data.summary)
      }
    }

```

```

if (data.recommendations !== undefined) {
  setRecommendations(data.recommendations)
}

if (data.lowStockItems !== undefined) {
  setLowStockItems(data.lowStockItems)
}

if (data.expiryAlerts !== undefined) {
  setExpiryAlerts(data.expiryAlerts)
}

if (data.myItems !== undefined) {
  setMyItems(data.myItems)
}

if (data.sellerProducts !== undefined) {
  setSellerProducts(data.sellerProducts)
}
}

const setDashboardErrorFromFailures = (failures) => {
  if (failures.length === 0) {
    setError("")
    return
  }

  if (failures.length === 5) {
    setError(
      getApiErrorMessage(
        failures[0].reason,
        'Unable to load dashboard data. Start the backend and login again.',
      ),
    )
    return
  }

  setError('Some dashboard sections are temporarily unavailable.')
}

useEffect(() => {
  let cancelled = false

  fetchDashboardData().then((data) => {
    if (cancelled) {
      return
    }

    applyDashboardData(data)
  })

```

```

    setDashboardErrorFromFailures(data.failures)
    setLoading(false)
  })

  return () => {
    cancelled = true
  }
}, [])

useEffect(() => {
  const sectionToOpen = location.state?.openSection

  if (!sectionToOpen) {
    return
  }

  setActiveSection(sectionToOpen)
  navigate(location.pathname, { replace: true, state: {} })
}, [location.pathname, location.state, navigate])

const recentItems = [...myItems].sort((first, second) => {
  if (first.lastPurchasedAt && second.lastPurchasedAt) {
    return new Date(second.lastPurchasedAt) - new Date(first.lastPurchasedAt)
  }

  if (second.lastPurchasedAt) {
    return 1
  }

  if (first.lastPurchasedAt) {
    return -1
  }

  return 0
}).slice(0, 6)

const insights = buildDashboardSmartInsights({
  summary,
  recommendations,
  lowStockItems,
  expiryAlerts,
  myItems,
})

const handleAcknowledgeExpiry = async (itemId) => {
  setAcknowledgingItemId(itemId)

  try {
    await acknowledgeExpiryAlert(itemId)
    setExpiryAlerts((current) => current.filter((item) => item.id !== itemId))
  }
})

```

```

        toast.success('Expiry reminder acknowledged')
      } catch (requestError) {
        setError(getApiErrorMessage(requestError, 'Could not acknowledge reminder right
now.'))
      } finally {
        setAcknowledgingItemId("")
      }
    }
  }

  if (loading) {
    return (
      <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
        {summaryCards.map((card) => (
          <SkeletonCard key={card.key} />
        ))}
      </div>
    )
  }

  return (
    <div className="space-y-8">
      <section className="rounded-3xl bg-gradient-to-r from-sky-600 via-cyan-500 to-emerald-400 px-6 py-8 text-white shadow-sm">
        <div className="flex items-center gap-3">
          <LayoutDashboard className="h-8 w-8" />
          <div>
            <p className="text-sm font-semibold uppercase tracking-[0.2em] text-white/80">
              User Dashboard
            </p>
            <h1 className="mt-1 text-3xl font-bold">Smart Grocery Overview</h1>
          </div>
        </div>
      </section>

      {error && (
        <div className="rounded-2xl border border-rose-200 bg-rose-50 px-4 py-3 text-sm text-rose-700">
          {error}
        </div>
      )}

      <section className="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
        {summaryCards.map((card) => {
          const Icon = card.icon
          return (
            <div
              key={card.key}
              className="rounded-3xl border border-slate-100 bg-white p-5 shadow-sm"
            >

```



```

        <div className={`inline-flex rounded-2xl bg-gradient-to-r p-3 text-white
${card.accent}`} >
          <Icon className="h-5 w-5" />
        </div>
        <p className="mt-4 text-sm font-semibold uppercase tracking-[0.15em] text-
slate-500">
          {card.label}
        </p>
        <h2 className="mt-2 text-3xl font-bold text-slate-900">
          {summary?.[card.key] ?? 0}
        </h2>
      </div>
    )
  }
}
</section>

<section className="grid gap-6 lg:grid-cols-[1.1fr_0.9fr]">
  <div className="space-y-4">
    <div>
      <p className="text-xs font-semibold uppercase tracking-[0.18em] text-sky-700">
        Smart Insights
      </p>
      <h3 className="mt-2 text-xl font-bold text-slate-900">
        Recommendations and Signals
      </h3>
    </div>

    <div className="grid gap-4 md:grid-cols-2">
      {insights.map((insight) => (
        <SmartInsightCard key={insight.id} {...insight} />
      ))}
    </div>
  </div>

  <div className="rounded-3xl border border-slate-100 bg-white p-5 shadow-sm">
    <p className="text-xs font-semibold uppercase tracking-[0.18em] text-sky-700">
      Navigation
    </p>
    <div className="mt-4 grid gap-3">
      {dashboardSections.map((section) => {
        const Icon = section.icon
        return (
          <button
            key={section.key}
            onClick={() => setActiveSection(section.key)}
            className="flex items-center justify-between rounded-2xl border border-slate-
100 px-4 py-3 text-left transition hover:border-sky-200 hover:bg-sky-50"
          >
            <div className="flex items-center gap-3">
              <Icon className="h-5 w-5 text-sky-700" />

```

```

        <div>
          <p className="font-semibold text-slate-900">{section.label}</p>
          <p className="text-sm text-slate-500">{section.sublabel}</p>
        </div>
      </div>
    </button>
  )
  }}}
</div>
</div>
</section>

<section className="grid gap-6 lg:grid-cols-2">
  <div className="rounded-3xl border border-slate-100 bg-white p-5 shadow-sm">
    <h3 className="text-xl font-bold text-slate-900">Low Stock Items</h3>
    <div className="mt-4 space-y-3">
      {lowStockItems.map((item) => (
        <div key={item.id} className="rounded-2xl border border-amber-100 bg-amber-
50 px-4 py-3">
          <div className="flex items-center justify-between">
            <div>
              <p className="font-semibold text-slate-900">{item.name}</p>
              <p className="text-sm text-slate-500">{item.category}</p>
            </div>
            <span className="text-sm font-bold text-amber-700">{item.quantity}
left</span>
          </div>
        </div>
      )})
    </div>
  </div>
  <div className="rounded-3xl border border-slate-100 bg-white p-5 shadow-sm">
    <h3 className="text-xl font-bold text-slate-900">Expiry Alerts</h3>
    <div className="mt-4 space-y-3">
      {expiryAlerts.map((item) => (
        <div key={item.id} className="rounded-2xl border border-rose-100 bg-rose-50
px-4 py-3">
          <div className="flex items-center justify-between gap-3">
            <div>
              <p className="font-semibold text-slate-900">{item.name}</p>
              <p className="text-sm text-slate-500">
                Expires on {formatExpiryDate(item.expiryDate)}
              </p>
            </div>
            <Button
              onClick={() => handleAcknowledgeExpiry(item.id)}
              disabled={acknowledgingItemId === item.id}
            >
              {acknowledgingItemId === item.id ? 'Saving...' : 'Acknowledge'}

```

```

        </Button>
      </div>
    </div>
  )}
</div>
</div>
</section>

<section className="rounded-3xl border border-slate-100 bg-white p-5 shadow-sm">
  <h3 className="text-xl font-bold text-slate-900">Recent Grocery Activity</h3>
  <div className="mt-4 space-y-3">
    {recentItems.map((item) => (
      <div key={item.id} className="rounded-2xl border border-slate-100 px-4 py-3">
        <div className="flex items-center justify-between">
          <div>
            <p className="font-semibold text-slate-900">{item.name}</p>
            <p className="text-sm text-slate-500">{item.category}</p>
          </div>
          <span className="text-sm text-slate-500">
            {item.lastPurchasedAt ? new Date(item.lastPurchasedAt).toLocaleString() : 'No
purchase date'}
          </span>
        </div>
      </div>
    )})}
  </div>
</section>
</div>
)
}

export default DashboardPage

```

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The Smart Grocery Management System is a successful implementation of a full-stack software engineering project focused on household inventory and grocery planning. The final system goes beyond a simple to-do list or shopping note by introducing structured inventory management, role-based access, expiry handling, shopping support, and administrative reporting.

The project demonstrates that even a common day-to-day problem can become a strong engineering case study when it is approached with proper system design. The frontend provides a user-friendly experience, the backend implements meaningful business rules, and the database stores information in a structured way that supports both daily operations and analytical views.

One of the most important achievements of the project is the smart behavior added on top of ordinary item storage. Low-stock detection, expiry-aware reminders, and shopping list generation convert static data into active support for the user. This significantly improves the quality and originality of the final year project.

The admin side of the application further strengthens the project by adding control, visibility, and reporting. This makes the system more complete, more scalable, and more relevant for academic evaluation under software engineering criteria.

Overall, the Smart Grocery Management System can be considered a practical, maintainable, and meaningful real-world software project.

7.2 Future Enhancements

The project can be improved further through the following future enhancements:

7.2.1 AI-Based Consumption Prediction

Machine learning or AI models can be used to predict future grocery needs based on user history and seasonal patterns.

7.2.2 Progressive Web Application Support

The current project is web-based. In future, it can be improved as a progressive web application so that it offers a stronger browser experience, better install-like behavior, and partial offline capability without changing the platform from web to mobile.

7.2.3 Barcode Scanning

Barcode scanning can simplify item entry and reduce manual typing effort.

7.2.4 Notification System

Push notifications, email reminders, or SMS alerts can notify users about low stock and expiry risks.

7.2.5 Advanced Analytics

Spending analysis, monthly category trends, and predictive restocking charts can be added to the reporting system.

7.2.6 Multi-User Household Sharing

Shared household workspaces can allow multiple family members to update the same grocery inventory collaboratively.

7.2.7 Seller or Supplier Integration

In future, grocery sourcing can be linked to supplier or marketplace systems to place orders directly from the shopping list.

7.2.8 Cloud Deployment

The project can be deployed on a cloud server with production configuration, external database hosting, and scalable security settings.

These future enhancements show that the current project is a strong foundation for continued development.

8. REFERENCES

1. Spring Boot Official Documentation. Available at: <https://spring.io/projects/spring-boot>
2. Spring Security Official Documentation. Available at: <https://spring.io/projects/spring-security>
3. Hibernate / JPA Documentation. Available at: <https://hibernate.org>
4. React Official Documentation. Available at: <https://react.dev>
5. Tailwind CSS Official Documentation. Available at: <https://tailwindcss.com>
6. MySQL Official Documentation. Available at: <https://dev.mysql.com/doc>
7. JWT Introduction and Standards Documentation. Available at: <https://jwt.io>
8. Axios Documentation. Available at: <https://axios-http.com>
9. Maven Documentation. Available at: <https://maven.apache.org>
10. ESLint Documentation. Available at: <https://eslint.org>
11. Project source code and implementation notes from the Smart Grocery repository developed during this project.